

MONOPOLY.LK

A Sri Lankan Monopoly Simulation - Strategic Edition

Complete Source Code Documentation
and Function Tree

No.	File	Lines	Functions
1	types.h	361	0
2	functions.h	184	0
3	main.c	17	1
4	board.c	205	5
5	players.c	405	15
6	finance.c	1076	33
7	game.c	640	14
8	events.c	402	6
9	economy.c	503	17
10	market.c	430	12
11	bank.c	394	12
12	auction.c	178	2
13	insurance.c	392	14

Total functions : 131

Call Tree (rooted at main)

Static call analysis: every call from a function body to another function in the project.

main

```

-> startGame
  -> determineTurnOrder
    -> resolveGroup
      -> resolveGroup (recursive)
      -> rollDice
  -> displayFinalResults
    -> calculateBuildingValue
    -> calculateNetWorth
      -> calculateBuildingValue
      -> calculatePropertyValue
        -> currentMarketValue
        -> modifierMultiplier
    -> calculatePropertyValue
      -> currentMarketValue
      -> modifierMultiplier
  -> determineWinner
    -> calculateNetWorth
      -> calculateBuildingValue
      -> calculatePropertyValue
        -> currentMarketValue
        -> modifierMultiplier
    -> formatLKR
  -> formatLKR
  -> initEconomy
  -> initMarket
  -> initializeBoard
    -> setProperty
    -> setRailway
    -> setSpecialSquare
    -> setUtility
  -> initializePlayers
    -> resetLoan
  -> playGame
    -> ageBuildings
    -> ageProperties
      -> currentMarketValue
      -> modifierMultiplier
    -> applyInflation
      -> applyRate
    -> countSolventPlayers
    -> decrementEventTimers
    -> decrementModifiers
    -> displayMarketConditions
      -> groupName
      -> printBoomDecline
      -> printSectionHeader
    -> printOtherConditions
      -> describeOtherCondition
        -> groupName
      -> printSectionHeader
    -> printRegionalSection
      -> describeRegionalCondition
        -> groupName
      -> printSectionHeader
    -> printSectionHeader
  -> displayRoundSummary
    -> calculateNetWorth
      -> calculateBuildingValue
      -> calculatePropertyValue
        -> currentMarketValue
        -> modifierMultiplier
    -> countHotels
    -> countHouses
    -> formatLKR
  -> drawRegionalCard
    -> addSourcedModifier
  -> enforceAntiSpeculation
    -> countUndevelopedProperties
    -> forceSellExcess
      -> countUndevelopedProperties
      -> sellUndevelopedPropertyToAuction
        -> adjustOwnedCount
        -> currentMarketValue

```

types.h

```

1  #ifndef TYPES_H
2  #define TYPES_H
3
4  #define MAX_PLAYERS      4
5  #define BOARD_SIZE      40
6  #define MAX_NAME_LENGTH 50
7  #define MAX_ROUNDS      500
8  #define START_MONEY     30000
9
10 #define GO_MONEY        2000
11 #define JAIL_BAIL       300
12
13 #define INCOME_TAX_RATE  15 /* base rate at game start, drifts with market */
14 #define COMMUNITY_FUND_TAX_RATE 10 /* base rate, drifts with market */
15
16 #define MAX_HOUSES      4
17 #define MAX_HOTELS      1
18
19 #define LOAN_DURATION_ROUNDS 20
20 #define LOAN_INTEREST_RATE  8 /* starting rate, drifts with inflation */
21 #define LOAN_COLLATERAL_PERCENT 75 /* Rule-LK 2: max loan is 75% of collateral */
22 #define LOAN_EXTEND_ROUNDS  10 /* Rule-LK 5: extension adds 10 rounds */
23 #define MORTGAGE_REPAY_PERCENT 110 /* redeeming a mortgage costs 110% of its value */
24
25 /* Loan interest rate limits */
26 #define LOAN_INTEREST_MIN  1 /* interest can never go below this */
27 #define LOAN_INTEREST_MAX  25 /* ...or above this */
28 #define LOAN_INTEREST_STEP 2 /* cards/regulations move it by 2 points */
29 #define TAX_RATE_MAX       25
30 #define HOTEL_LUXURY_TAX_PERCENT 25 /* luxury tax: 25% of a hotel's value */
31
32 /* Strategy decision thresholds, in LKR (rule-book strategy rules) */
33 #define MIN_CASH_SAFETY    1000 /* reserve before spending / borrowing */
34 #define MIN_CASH_FLOOR     500 /* risk taker's floor before mortgaging */
35 #define TRADER_CASH_FLOOR 1200 /* opportunistic trader's mortgage floor */
36 #define COMFORTABLE_CASH   3000 /* conservative banker's comfort line */
37 #define BAIL_CASH_SPARE    1500 /* conservative banker's bail threshold */
38 #define RICH_CASH_LINE     5000 /* rich-player threshold */
39 #define EXPENSIVE_PROP_PRICE 6000 /* price above which a trader insures */
40
41 /* Rent tables (Tables 7, 8): one railway/utility owned -> this rent */
42 #define RAILWAY_RENT_1     250
43 #define RAILWAY_RENT_2     500
44 #define RAILWAY_RENT_3     1000
45 #define RAILWAY_RENT_4     2000
46 #define UTILITY_RENT_ONE   4 /* rent = 4 x dice */
47 #define UTILITY_RENT_TWO  10 /* rent = 10 x dice when both owned */
48
49 /* Auction rules */
50 #define AUCTION_OPENING_DIVISOR 2 /* opening bid = asking value / 2 */
51 #define AUCTION_BID_INCREMENT 250 /* every bid must raise the price by this */
52 #define AUCTION_MAX_ROUNDS 200 /* safety limit so bidding always ends */
53
54 /* Property market review (Rule-LK 30) */
55 #define MARKET_REVIEW_EVERY 10 /* review happens every 10 rounds */
56 #define MARKET_COOLDOWN_ROUNDS 30 /* a group can't re-trigger for 30 rounds */
57
58 /* Property ageing and neglect (Rule-LK 15, 16, 25-27) */
59 #define PROPERTY_AGE_LIMIT 50 /* value starts dropping past this age */
60 #define DEPRECIATION_STEP 5 /* every 5 rounds past the limit */
61 #define MAX_DEPRECIATION 30
62 #define RENOVATION_COST_PERCENT 10 /* renovation costs 10% of current value */
63 #define RENOVATION_RENT_BOOST 5 /* renovated base rent rises 5% */
64 #define NEGLECT_ROUNDS 20 /* no maintenance this long = structural damage */
65 #define STRUCTURAL_VALUE_LOSS 15 /* damaged building loses 15% of its value */
66 #define STRUCTURAL_RENT_LOSS 25 /* ...and 25% of its rent */
67 #define DAMAGE_REPAIR_MULTIPLIER 150 /* fixing structural damage costs 150% */
68
69 /* Insurance rules */
70 #define REPAIR_COST_PERCENT 30 /* repairs cost 30% of a property's value */
71 #define PREMIUM_BASIC_PERCENT 5
72 #define PREMIUM_COMPREHENSIVE_PERCENT 10
73 #define PREMIUM_INTERRUPTION_PERCENT 15
74 #define INSURANCE_DURATION_ROUNDS 20
75 #define INSURANCE_RENEW_AT 10 /* renew when 10 or fewer rounds remain */
76 #define INSURANCE_EXPIRY_WARNING 3 /* warn 3 rounds before a policy expires */
77 #define DISASTER_WEIGHT 20 /* every disaster is equally likely at base */
78 #define DISASTER_BOOST 30 /* weather/unrest makes one type +30 likely */
79 #define BI_LOST_INCOME_ROUNDS 5 /* Business Interruption: no rent for 5 rounds */
80

```

```
81  /* Jail (Rule 13) */
82  #define JAIL_SQUARE          10
83  #define JAIL_MAX_TURNS      3
84
85  /* Event / card deck sizes */
86  #define EVENT_CARD_COUNT     20
87  #define ECONOMIC_EVENT_COUNT 8
88  #define REGULATION_COUNT     8
89  #define REGIONAL_CARD_COUNT  12
90
91  /* Round cadences for periodic events */
92  #define ECONOMY_CYCLE        10 /* disasters, inflation, market review */
93  #define EVENT_CYCLE          15 /* economic events and regional cards */
94  #define REGULATION_CYCLE     20 /* government regulations */
95
96  /* Anti-Speculation Act (Rule-LK 8) */
97  #define MAX_UNDEVELOPED_PROPS 3 /* at most this many undeveloped properties */
98  #define ANTI_SPEC_TRIGGER_ROUNDS 5 /* enforced after 5 consecutive rounds */
99
100 /* Board Square Types */
101 typedef enum
102 {
103     GO,
104     PROPERTY,
105     RAILWAY,
106     UTILITY,
107     TAX,
108     COMMUNITY_FUND,
109     EVENT,
110     BANK,
111     INSURANCE,
112     JAIL,
113     FREE_PARKING,
114     GO_TO_JAIL
115 } SquareType;
116
117
118 /* Property Colour Groups */
119 typedef enum
120 {
121     BROWN,
122     LIGHT_BLUE,
123     PINK,
124     ORANGE,
125     RED,
126     YELLOW,
127     GREEN,
128     DARK_BLUE,
129     NO_GROUP
130 } PropertyGroup;
131
132
133 /* Player Strategies */
134 typedef enum
135 {
136     AGGRESSIVE_INVESTOR,
137     CONSERVATIVE_BANKER,
138     RISK_TAKER,
139     OPPORTUNISTIC_TRADER
140 } PlayerType;
141
142
143 /* Insurance Types */
144 typedef enum
145 {
146     NO_INSURANCE,
147     BASIC_INSURANCE,
148     COMPREHENSIVE_INSURANCE,
149     BUSINESS_INTERRUPTION
150 } InsuranceType;
151
152
153 /* Disaster Types (Rule-LK 10) */
154 typedef enum
155 {
156     FIRE,
157     FLOOD,
158     RIOT,
```

```
163     BUILDING_COLLAPSE,
164     ELECTRICAL_FAILURE
165 } DisasterType;
166
167
168
169 typedef struct
170 {
171     int active;
172     int amount;
173     int interestRate;
174     int remainingRounds;
175
176 } Loan;
177
178
179 typedef struct
180 {
181     char name[MAX_NAME_LENGTH];
182
183     PropertyGroup group;
184
185     int purchasePrice;
186     int mortgageValue;
187     int baseRent;
188
189     int houseCost;
190     int hotelCost;
191
192     int owner;          /* -1 means unowned */
193
194     int houses;
195     int hotel;
196
197     int mortgaged;
198     int loanLocked;     /* pledged as loan collateral */
199
200     InsuranceType insurance;
201     int insuranceRoundsLeft;
202
203     int damaged;        /* disaster damage - no rent until repaired */
204     int repairCostOwed;
205     int lostIncomeRoundsLeft; /* Business Interruption: no rent for 5 rounds */
206
207     int age;
208     int depreciation;
209
210     int condition;      /* building condition, Rule-LK 25-27 */
211     int roundsSinceMaintenance;
212     int structurallyDamaged;
213     int preDamagePurchasePrice; /* remembered so renovation can restore it */
214     int preDamageBaseRent;
215     int maintenanceCostMultiplierPercent;
216
217 } Property;
218
219
220 typedef struct
221 {
222     int index;
223
224     SquareType type;
225
226     char name[MAX_NAME_LENGTH];
227
228     Property property;
229
230 } Square;
231
232
233 typedef struct
234 {
235     char name[MAX_NAME_LENGTH];
236
237     PlayerType strategy;
238
239     int position;
240
241     int cash;
242
243     int inJail;
244
```

```

245     int jailTurns;
246
247     int bankrupt;
248
249     int propertiesOwned;
250
251     int railwaysOwned;
252
253     int utilitiesOwned;
254
255     Loan loan;
256
257     int sufferedLoss; /* Risk Taker only insures after a loss */
258
259     int antiSpecRounds; /* rounds in a row above the 3-undeveloped limit */
260
261 } Player;
262
263
264 /* Kinds of temporary modifiers that can be active at once. Each
265 event, card, regulation, market review and regional card adds a
266 timed entry to Economy.modifiers[]. All matching entries are
267 multiplied together (Rule-LK 34) so effects are cumulative and
268 never permanently corrupt the base property values/costs. */
269 typedef enum
270 {
271     MOD_VALUE_GLOBAL, /* all property values */
272     MOD_GROUP_VALUE, /* one colour group's property values */
273     MOD_INDEX_VALUE, /* a single square's value (any type) */
274     MOD_RAIL_VALUE, /* all railway station values */
275     MOD_PURCHASE_PRICE, /* direct purchase prices (boom +15%) */
276     MOD_RENT_GLOBAL, /* all property/rail/utility rent */
277     MOD_GROUP_RENT, /* one colour group's rent */
278     MOD_HOTEL_RENT, /* hotel rent */
279     MOD_RAIL_RENT, /* railway rent */
280     MOD_UTIL_RENT, /* utility rent */
281     MOD_CONSTRUCTION, /* house/hotel construction costs */
282     MOD_INSURANCE, /* insurance premiums */
283     MOD_MARKET_MORTGAGE, /* mortgage values (boom +15% / decline -10%) */
284     MOD_AUCTION_PRICE, /* auction starting prices (decline -25%) */
285     MOD_FLOOD_RISK, /* Heavy Monsoon - flood more likely */
286     MOD_RIOT_RISK, /* Political Unrest - riot more likely */
287     MOD_BI_CLAIMS, /* Political Unrest - business interruption claims */
288     MOD RECESSION /* Economic Recession - strategies may react */
289
290 } ModifierType;
291
292
293 /* Where a modifier came from - used by Rule-LK 36 to group the
294 currently active conditions into Market Boom / Market Decline /
295 Regional Development sections. */
296 typedef enum
297 {
298     SRC_GENERAL, /* events, national cards, government regulations */
299     SRC_BOOM, /* Property Market Review - boom group */
300     SRC_DECLINE, /* Property Market Review - decline group */
301     SRC_REGIONAL /* Regional Development Card */
302
303 } ModifierSource;
304
305
306 typedef struct
307 {
308     ModifierType type;
309     int group; /* group for GROUP modifiers, else -1 */
310     int index; /* square index for INDEX modifiers, else -1 */
311     int percent; /* 100 means no change */
312     int roundsLeft;
313     ModifierSource source;
314
315 } ActiveModifier;
316
317
318 #define MAX_MODIFIERS 48
319
320
321 /* Current state of the economy - inflation, interest rate, and every
322 temporary rent/cost bonus or penalty from events and the market. */
323 typedef struct
324 {
325     int inflationRate;
326     int loanInterestRate;

```

```
327
328     int constructionSuspendedRoundsLeft;
329
330     int closedPropertyIndex;
331     int closedPropertyRoundsLeft;
332
333     int incomeTaxRate;
334
335     int antiSpeculationActive;
336
337     int currentCardIndex; /* position in the National Event Card deck */
338
339     int groupCooldownUntilRound[NO_GROUP];
340
341     int lastBoomGroup;
342     int lastDeclineGroup;
343
344     ActiveModifier modifiers[MAX_MODIFIERS];
345     int modifierCount;
346
347 } Economy;
348
349
350 /* Everything the game needs to remember, bundled into one struct so
351    it can be passed to functions as a plain array parameter instead
352    of using global variables or pointers. */
353 typedef struct
354 {
355     Square board[BOARD_SIZE];
356     Player players[MAX_PLAYERS];
357     Economy economy;
358
359 } GameState;
360
361 #endif
```

functions.h

```

1  #ifndef FUNCTIONS_H
2  #define FUNCTIONS_H
3
4  #include "types.h"
5
6  /* board.c */
7
8  void initializeBoard(GameState game[]);
9
10 void setProperty(GameState game[],
11                 int index,
12                 char name[],
13                 PropertyGroup group,
14                 int purchasePrice,
15                 int baseRent,
16                 int mortgageValue,
17                 int houseCost,
18                 int hotelCost);
19
20 void setRailway(GameState game[], int index, char name[], int purchasePrice);
21 void setUtility(GameState game[], int index, char name[], int purchasePrice);
22
23 void setSpecialSquare(GameState game[],
24                      int index,
25                      SquareType type,
26                      char name[]);
27
28 /* players.c */
29
30 void initializePlayers(GameState game[]);
31
32 int shouldBuyProperty(GameState game[], int playerIndex);
33 int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel);
34 int wantsLoan(GameState game[], int playerIndex);
35 int wantsToRepayLoan(GameState game[], int playerIndex);
36 int wantsIncreaseLoan(GameState game[], int playerIndex);
37 int wantsExtendLoan(GameState game[], int playerIndex);
38 int wantsRefinance(GameState game[], int playerIndex);
39 InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex);
40 int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent);
41 int shouldMaintain(GameState game[], int playerIndex, int condition, int cost);
42 int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid);
43 int shouldMortgage(GameState game[], int playerIndex);
44 int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost);
45 int shouldPayBail(GameState game[], int playerIndex);
46
47 /* finance.c */
48
49 void stripOwnership(GameState game[], int index);
50 void resetLoan(GameState game[], int playerIndex);
51 void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta);
52 PropertyGroup groupOf(GameState game[], int index);
53
54 void buyProperty(GameState game[], int playerIndex);
55 void payRent(GameState game[], int playerIndex, int diceValue);
56 void payTax(GameState game[], int playerIndex);
57 void payCommunityFundTax(GameState game[], int playerIndex);
58 void payMoney(GameState game[], int playerIndex, int amount);
59 void liquidateBankruptAssets(GameState game[], int playerIndex);
60 void receiveMoney(GameState game[], int playerIndex, int amount);
61
62 int calculatePropertyValue(GameState game[], int playerIndex);
63 int calculateBuildingValue(GameState game[], int playerIndex);
64 int calculateNetWorth(GameState game[], int playerIndex);
65
66 int findPropertyToMortgage(GameState game[], int playerIndex);
67 int findMortgagedProperty(GameState game[], int playerIndex);
68 void mortgageProperty(GameState game[], int playerIndex);
69 void redeemMortgage(GameState game[], int playerIndex);
70 void handleMortgageDecisions(GameState game[], int playerIndex);
71
72 int calculateRent(GameState game[], int position);
73 int calculateRailwayRent(GameState game[], int playerIndex);
74 int calculateUtilityRent(GameState game[], int playerIndex, int diceValue);
75 int countUndevelopedProperties(GameState game[], int playerIndex);
76
77 int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos);
78 void sellLowValueProperty(GameState game[], int playerIndex);
79 void sellDecliningProperties(GameState game[], int playerIndex);
80 void enforceAntiSpeculation(GameState game[]);

```

```
81 void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex);
82
83 int groupSize(GameState game[], PropertyGroup group);
84 int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group);
85 int developGroup(GameState game[], int playerIndex, PropertyGroup group);
86 void constructBuildings(GameState game[], int playerIndex);
87
88 /* game.c */
89
90 void startGame(GameState game[]);
91 void playGame(GameState game[], int turnOrder[]);
92 int playTurn(GameState game[], int playerIndex);
93 int movePlayer(GameState game[], int playerIndex, int dice);
94 int rollDice(void);
95 int handleJail(GameState game[], int playerIndex);
96
97 int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
98                 int output[], int startIndex);
99
100 void determineTurnOrder(GameState game[], int turnOrder[]);
101 void displayRoundSummary(GameState game[], int round);
102 int countSolventPlayers(GameState game[]);
103 int countHouses(GameState game[], int playerIndex);
104 int countHotels(GameState game[], int playerIndex);
105 int determineWinner(GameState game[]);
106 void displayFinalResults(GameState game[]);
107
108 /* events.c */
109
110 void clampLoanInterest(GameState game[]);
111 void adjustLoanInterest(GameState game[], int deltaPercent);
112 void executeEvent(GameState game[], int playerIndex);
113 void triggerEconomicEvent(GameState game[]);
114 void triggerGovernmentRegulation(GameState game[]);
115 void decrementEventTimers(GameState game[]);
116
117 /* market.c */
118
119 void initMarket(GameState game[]);
120 void groupName(PropertyGroup group, char buffer[]);
121 PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2);
122 void reviewPropertyMarket(GameState game[], int currentRound);
123 void drawRegionalCard(GameState game[]);
124 void displayMarketConditions(GameState game[]);
125
126 /* bank.c */
127
128 int groupBasePrice(GameState game[], PropertyGroup group);
129 int totalEligibleCollateral(GameState game[], int playerIndex);
130 int calculateMaxLoan(GameState game[], int playerIndex);
131 void obtainLoan(GameState game[], int playerIndex);
132 void repayLoan(GameState game[], int playerIndex, int amount);
133 void handleBankVisit(GameState game[], int playerIndex);
134 void increaseLoan(GameState game[], int playerIndex);
135 void extendLoan(GameState game[], int playerIndex);
136 void refinanceLoan(GameState game[], int playerIndex);
137 void demolishBuildingsOn(GameState game[], int index);
138 void foreclose(GameState game[], int playerIndex);
139 void processLoans(GameState game[]);
140
141 /* insurance.c */
142
143 int propertyValue(GameState game[], int propIndex);
144 int repairCost(GameState game[], int propIndex);
145 DisasterType pickDisaster(GameState game[]);
146 int isCovered(InsuranceType policy, DisasterType disaster);
147 int compensationPercent(InsuranceType policy);
148 void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type);
149 int findPropertyToInsure(GameState game[], int playerIndex);
150 int findPropertyToRenew(GameState game[], int playerIndex);
151 void handleInsuranceVisit(GameState game[], int playerIndex);
152 void tryAutoRepair(GameState game[], int playerIndex);
153 void triggerDisaster(GameState game[]);
154 void processInsuranceExpiry(GameState game[]);
155
156 /* economy.c */
157
158 int applyRate(int oldValue, int ratePercent);
159 void initEconomy(GameState game[]);
160 void applyInflation(GameState game[]);
161 void addModifier(GameState game[], ModifierType type, int group, int index,
162                 int percent, int roundsLeft);
```

```
163 void addSourcedModifier(GameState game[], ModifierType type, int group,
164                          int index, int percent, int roundsLeft,
165                          ModifierSource source);
166 int modifierMultiplier(GameState game[], ModifierType type, int group, int index);
167 int isModifierActive(GameState game[], ModifierType type, int group, int index);
168 void decrementModifiers(GameState game[]);
169 void formatLKR(int amount, char out[]);
170 int currentMarketValue(GameState game[], int propIndex);
171 int currentTaxRatePercent(GameState game[], int baseRate);
172 void ageProperties(GameState game[]);
173 void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex);
174 int rentConditionPercent(int condition);
175 void ageBuildings(GameState game[]);
176 void performMaintenance(GameState game[], int playerIndex);
177 void renovateStructuralDamage(GameState game[], int playerIndex);
178
179 /* auction.c */
180
181 int getAskingValue(GameState game[], int propIndex);
182 void runAuction(GameState game[], int propIndex, int sellerIndex);
183
184 #endif
```

main.c

```
1  #include <stdio.h>
2  #include "types.h"
3  #include "functions.h"
4
5  int main(void)
6  {
7      /* the only place a GameState is ever created */
8      GameState game[1] = {0};
9
10     printf("=====\n");
11     printf("    MONOPOLY-LK SIMULATION\n");
12     printf("=====\n\n");
13
14     startGame(game);
15
16     return 0;
17 }
```

board.c

```

1  #include <stdio.h>
2  #include <string.h>
3  #include "types.h"
4  #include "functions.h"
5
6  void setProperty(GameState game[],
7                  int index,
8                  char name[],
9                  PropertyGroup group,
10                 int purchasePrice,
11                 int baseRent,
12                 int mortgageValue,
13                 int houseCost,
14                 int hotelCost)
15 {
16     game[0].board[index].index = index;
17     game[0].board[index].type = PROPERTY;
18
19     strcpy(game[0].board[index].name, name);
20     strcpy(game[0].board[index].property.name, name);
21
22     game[0].board[index].property.group = group;
23     game[0].board[index].property.purchasePrice = purchasePrice;
24     game[0].board[index].property.baseRent = baseRent;
25     game[0].board[index].property.mortgageValue = mortgageValue;
26     game[0].board[index].property.houseCost = houseCost;
27     game[0].board[index].property.hotelCost = hotelCost;
28
29     /* nobody owns it yet */
30     game[0].board[index].property.owner = -1;
31
32     game[0].board[index].property.houses = 0;
33     game[0].board[index].property.hotel = 0;
34     game[0].board[index].property.mortgaged = 0;
35     game[0].board[index].property.loanLocked = 0;
36
37     game[0].board[index].property.insurance = NO_INSURANCE;
38     game[0].board[index].property.insuranceRoundsLeft = 0;
39
40     game[0].board[index].property.damaged = 0;
41     game[0].board[index].property.repairCostOwed = 0;
42     game[0].board[index].property.lostIncomeRoundsLeft = 0;
43
44     game[0].board[index].property.age = 0;
45     game[0].board[index].property.depreciation = 0;
46     game[0].board[index].property.condition = 100;
47
48     game[0].board[index].property.roundsSinceMaintenance = 0;
49     game[0].board[index].property.structurallyDamaged = 0;
50     game[0].board[index].property.preDamagePurchasePrice = 0;
51     game[0].board[index].property.preDamageBaseRent = 0;
52     game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
53 }
54
55 void setRailway(GameState game[], int index, char name[], int purchasePrice)
56 {
57     game[0].board[index].index = index;
58     game[0].board[index].type = RAILWAY;
59
60     strcpy(game[0].board[index].name, name);
61     strcpy(game[0].board[index].property.name, name);
62
63     game[0].board[index].property.purchasePrice = purchasePrice;
64     game[0].board[index].property.mortgageValue = purchasePrice / 2;
65     game[0].board[index].property.owner = -1;
66     game[0].board[index].property.mortgaged = 0;
67     game[0].board[index].property.loanLocked = 0;
68
69     /* railways can't have houses/hotels/insurance, but we still zero
70        these out since GameState is local now and won't do it for us */
71     game[0].board[index].property.houses = 0;
72     game[0].board[index].property.hotel = 0;
73     game[0].board[index].property.insurance = NO_INSURANCE;
74     game[0].board[index].property.insuranceRoundsLeft = 0;
75     game[0].board[index].property.damaged = 0;
76     game[0].board[index].property.repairCostOwed = 0;
77     game[0].board[index].property.lostIncomeRoundsLeft = 0;
78     game[0].board[index].property.age = 0;
79     game[0].board[index].property.depreciation = 0;
80     game[0].board[index].property.condition = 100;

```

```

81     game[0].board[index].property.roundsSinceMaintenance = 0;
82     game[0].board[index].property.structurallyDamaged = 0;
83     game[0].board[index].property.preDamagePurchasePrice = 0;
84     game[0].board[index].property.preDamageBaseRent = 0;
85     game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
86     game[0].board[index].property.baseRent = 0;
87     game[0].board[index].property.houseCost = 0;
88     game[0].board[index].property.hotelCost = 0;
89 }
90
91 void setUtility(GameState game[], int index, char name[], int purchasePrice)
92 {
93     game[0].board[index].index = index;
94     game[0].board[index].type = UTILITY;
95
96     strcpy(game[0].board[index].name, name);
97     strcpy(game[0].board[index].property.name, name);
98
99     game[0].board[index].property.purchasePrice = purchasePrice;
100    game[0].board[index].property.mortgageValue = purchasePrice / 2;
101    game[0].board[index].property.owner = -1;
102    game[0].board[index].property.mortgaged = 0;
103    game[0].board[index].property.loanLocked = 0;
104
105    game[0].board[index].property.houses = 0;
106    game[0].board[index].property.hotel = 0;
107    game[0].board[index].property.insurance = NO_INSURANCE;
108    game[0].board[index].property.insuranceRoundsLeft = 0;
109    game[0].board[index].property.damaged = 0;
110    game[0].board[index].property.repairCostOwed = 0;
111    game[0].board[index].property.lostIncomeRoundsLeft = 0;
112    game[0].board[index].property.age = 0;
113    game[0].board[index].property.depreciation = 0;
114    game[0].board[index].property.condition = 100;
115    game[0].board[index].property.roundsSinceMaintenance = 0;
116    game[0].board[index].property.structurallyDamaged = 0;
117    game[0].board[index].property.preDamagePurchasePrice = 0;
118    game[0].board[index].property.preDamageBaseRent = 0;
119    game[0].board[index].property.maintenanceCostMultiplierPercent = 100;
120    game[0].board[index].property.baseRent = 0;
121    game[0].board[index].property.houseCost = 0;
122    game[0].board[index].property.hotelCost = 0;
123 }
124
125 void setSpecialSquare(GameState game[],
126                      int index,
127                      SquareType type,
128                      char name[])
129 {
130     game[0].board[index].index = index;
131     game[0].board[index].type = type;
132
133     strcpy(game[0].board[index].name, name);
134
135     /* Owner must be -1 (unowned), not the default 0, otherwise this
136        square would look like it belongs to Player 0. */
137     game[0].board[index].property.owner = -1;
138     game[0].board[index].property.mortgaged = 0;
139     game[0].board[index].property.loanLocked = 0;
140     game[0].board[index].property.houses = 0;
141     game[0].board[index].property.hotel = 0;
142     game[0].board[index].property.insurance = NO_INSURANCE;
143 }
144
145 void initializeBoard(GameState game[])
146 {
147     setSpecialSquare(game, 0, GO, "GO");
148     setSpecialSquare(game, 2, COMMUNITY_FUND, "Community Development Fund");
149     setSpecialSquare(game, 4, TAX, "Income Tax");
150     setSpecialSquare(game, 7, EVENT, "National Event Card");
151     setSpecialSquare(game, 10, JAIL, "Jail / Just Visiting");
152     setSpecialSquare(game, 17, INSURANCE, "Sri Lanka Insurance");
153     setSpecialSquare(game, 20, FREE_PARKING, "Free Parking");
154     setSpecialSquare(game, 22, EVENT, "National Event Card");
155     setSpecialSquare(game, 30, GO_TO_JAIL, "Go To Jail");
156     setSpecialSquare(game, 33, INSURANCE, "Ceylinco Insurance");
157     setSpecialSquare(game, 36, EVENT, "National Event Card");
158     setSpecialSquare(game, 38, BANK, "Bank of Ceylon");
159
160     setRailway(game, 5, "Colombo Fort Railway Station", 1500);
161     setRailway(game, 15, "Kandy Railway Station", 1500);
162     setRailway(game, 25, "Galle Railway Station", 1500);

```

```
163     setRailway(game, 35, "Jaffna Railway Station", 1500);
164
165     setUtility(game, 12, "Ceylon Electricity Board", 1500);
166     setUtility(game, 28, "National Water Supply and Drainage Board", 1500);
167
168     /* Brown */
169     setProperty(game, 1, "Pettah", BROWN, 1500, 100, 750, 500, 2000);
170     setProperty(game, 3, "Maradana", BROWN, 1800, 120, 900, 500, 2000);
171
172     /* Light Blue */
173     setProperty(game, 6, "Bambalapitiya", LIGHT_BLUE, 2500, 180, 1250, 750, 3000);
174     setProperty(game, 8, "Wellawatte", LIGHT_BLUE, 2700, 200, 1350, 750, 3000);
175     setProperty(game, 9, "Mount Lavinia", LIGHT_BLUE, 3000, 220, 1500, 750, 3000);
176
177     /* Pink */
178     setProperty(game, 11, "Nugegoda", PINK, 3500, 260, 1750, 1000, 4000);
179     setProperty(game, 13, "Maharagama", PINK, 3800, 280, 1900, 1000, 4000);
180     setProperty(game, 14, "Kottawa", PINK, 4000, 300, 2000, 1000, 4000);
181
182     /* Orange */
183     setProperty(game, 16, "Negombo", ORANGE, 4500, 350, 2250, 1250, 5000);
184     setProperty(game, 18, "Katunayake", ORANGE, 4700, 370, 2350, 1250, 5000);
185     setProperty(game, 19, "Ja-Ela", ORANGE, 5000, 400, 2500, 1250, 5000);
186
187     /* Red */
188     setProperty(game, 21, "Kandy City", RED, 5500, 450, 2750, 1500, 6000);
189     setProperty(game, 23, "Peradeniya", RED, 5800, 480, 2900, 1500, 6000);
190     setProperty(game, 24, "Katugastota", RED, 6000, 500, 3000, 1500, 6000);
191
192     /* Yellow */
193     setProperty(game, 26, "Galle Fort", YELLOW, 6500, 600, 3250, 2000, 8000);
194     setProperty(game, 27, "Unawatuna", YELLOW, 6800, 620, 3400, 2000, 8000);
195     setProperty(game, 29, "Hikkaduwa", YELLOW, 7000, 650, 3500, 2000, 8000);
196
197     /* Green */
198     setProperty(game, 31, "Jaffna Town", GREEN, 8000, 750, 4000, 2500, 10000);
199     setProperty(game, 32, "Nallur", GREEN, 8300, 780, 4150, 2500, 10000);
200     setProperty(game, 34, "Trincomalee", GREEN, 8500, 800, 4250, 2500, 10000);
201
202     /* Dark Blue */
203     setProperty(game, 37, "Nuwara Eliya", DARK_BLUE, 10000, 1000, 5000, 3000, 12000);
204     setProperty(game, 39, "Galle Face", DARK_BLUE, 12000, 1200, 6000, 3000, 12000);
205 }
```

players.c

```

1  #include <stdio.h>
2  #include <string.h>
3  #include "types.h"
4  #include "functions.h"
5
6  void initializePlayers(GameState game[])
7  {
8      int i;
9
10     strcpy(game[0].players[0].name, "Aggressive Investor");
11     game[0].players[0].strategy = AGGRESSIVE_INVESTOR;
12
13     strcpy(game[0].players[1].name, "Conservative Banker");
14     game[0].players[1].strategy = CONSERVATIVE_BANKER;
15
16     strcpy(game[0].players[2].name, "Risk Taker");
17     game[0].players[2].strategy = RISK_TAKER;
18
19     strcpy(game[0].players[3].name, "Opportunistic Trader");
20     game[0].players[3].strategy = OPPORTUNISTIC_TRADER;
21
22     for(i = 0; i < MAX_PLAYERS; i++)
23     {
24         game[0].players[i].position = 0;
25         game[0].players[i].cash = START_MONEY;
26
27         game[0].players[i].inJail = 0;
28         game[0].players[i].jailTurns = 0;
29
30         game[0].players[i].bankrupt = 0;
31
32         game[0].players[i].propertiesOwned = 0;
33         game[0].players[i].railwaysOwned = 0;
34         game[0].players[i].utilitiesOwned = 0;
35
36         resetLoan(game, i);
37
38         game[0].players[i].sufferedLoss = 0;
39         game[0].players[i].antiSpecRounds = 0;
40     }
41 }
42
43 /* Decides whether a player wants to buy the property they just
44    landed on. Each strategy has its own rule, from Section 3. */
45 int shouldBuyProperty(GameState game[], int playerIndex)
46 {
47     int price;
48     int position;
49
50     position = game[0].players[playerIndex].position;
51     price = game[0].board[position].property.purchasePrice;
52
53     /* match the price buyProperty() actually charges (Rule-LK 31/34) */
54     if(game[0].board[position].type == PROPERTY)
55     {
56         price = (currentMarketValue(game, position) *
57                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
58                                     game[0].board[position].property.group, -1)) / 100;
59     }
60
61     switch(game[0].players[playerIndex].strategy)
62     {
63     case AGGRESSIVE_INVESTOR:
64         /* buys to complete a monopoly group even if it dips below
65            the usual 1000 reserve (Section 3.1) */
66         if(wouldCompleteMonopoly(game, playerIndex, position))
67             return 1;
68         return (game[0].players[playerIndex].cash - price >= 1000);
69
70     case CONSERVATIVE_BANKER:
71         /* avoids property purchases entirely during a recession,
72            but will still buy rail/utility income (Section 3.2) */
73         if(game[0].board[position].type == PROPERTY &&
74            isModifierActive(game, MOD_RECESSION, -1, -1))
75             return 0;
76         return (game[0].players[playerIndex].cash - price >=
77                 game[0].players[playerIndex].cash / 2);
78
79     case RISK_TAKER:
80         return (game[0].players[playerIndex].cash >= price);

```

```

81
82     case OPPORTUNISTIC_TRADER:
83         /* never buys into a group that is currently declining */
84         if(game[0].board[position].type == PROPERTY &&
85             modifierMultiplier(game, MOD_GROUP_VALUE,
86                 game[0].board[position].property.group, -1) < 100)
87             return 0;
88         /* only buys cheaper properties */
89         return (price <= game[0].players[playerIndex].cash * 40 / 100);
90
91     default:
92         return 0;
93 }
94 }
95
96 /* Decides whether a player is willing to spend on a house/hotel. */
97 int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
98 {
99     int cash;
100
101     cash = game[0].players[playerIndex].cash;
102
103     switch(game[0].players[playerIndex].strategy)
104     {
105         case AGGRESSIVE_INVESTOR:
106             return (cash - cost >= MIN_CASH_SAFETY);
107
108         case CONSERVATIVE_BANKER:
109             /* never builds hotels while a loan is active - houses are
110              still fine (Section 3.2) */
111             if(game[0].players[playerIndex].loan.active && isHotel)
112                 return 0;
113             return (cash - cost >= cash / 2);
114
115         case RISK_TAKER:
116             return (cash >= cost);
117
118         case OPPORTUNISTIC_TRADER:
119             /* builds eagerly while a construction subsidy is active */
120             if(modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100)
121                 return (cash - cost >= cash * 30 / 100);
122
123             /* delays building while inflation pushes costs up */
124             if(game[0].economy.inflationRate > 0)
125                 return (cash - cost >= cash * 60 / 100);
126
127             return (cash - cost >= cash * 40 / 100);
128
129     default:
130         return 0;
131     }
132 }
133
134 int wantsLoan(GameState game[], int playerIndex)
135 {
136     int cash;
137
138     cash = game[0].players[playerIndex].cash;
139
140     switch(game[0].players[playerIndex].strategy)
141     {
142         case AGGRESSIVE_INVESTOR:
143             return 1;
144
145         case CONSERVATIVE_BANKER:
146             /* only borrows if bankruptcy is close */
147             return (cash < MIN_CASH_SAFETY);
148
149         case RISK_TAKER:
150             return 1;
151
152         case OPPORTUNISTIC_TRADER:
153             return (cash < RICH_CASH_LINE);
154
155     default:
156         return 0;
157     }
158 }
159
160 int wantsToRepayLoan(GameState game[], int playerIndex)
161 {
162     int cash;

```

```

163     int loanAmount;
164
165     cash = game[0].players[playerIndex].cash;
166     loanAmount = game[0].players[playerIndex].loan.amount;
167
168     switch(game[0].players[playerIndex].strategy)
169     {
170         case AGGRESSIVE_INVESTOR:
171             return (cash >= 2 * loanAmount);
172
173         case CONSERVATIVE_BANKER:
174             return (cash >= loanAmount);
175
176         case RISK_TAKER:
177             /* rarely repays - prefers to keep using the cash */
178             return (cash >= 5 * loanAmount);
179
180         case OPPORTUNISTIC_TRADER:
181             return (cash >= (loanAmount * 3) / 2);
182
183         default:
184             return 0;
185     }
186 }
187
188 /* Rule-LK 5: whether a player wants to borrow more against new
189 collateral during a Bank visit. */
190 int wantsIncreaseLoan(GameState game[], int playerIndex)
191 {
192     int cash;
193
194     cash = game[0].players[playerIndex].cash;
195
196     switch(game[0].players[playerIndex].strategy)
197     {
198         case AGGRESSIVE_INVESTOR:
199             return 1;
200
201         case CONSERVATIVE_BANKER:
202             return (cash < COMFORTABLE_CASH);
203
204         case RISK_TAKER:
205             return 1;
206
207         case OPPORTUNISTIC_TRADER:
208             return (cash < RICH_CASH_LINE);
209
210         default:
211             return 0;
212     }
213 }
214
215 /* Rule-LK 5: whether a player wants to extend the loan term. */
216 int wantsExtendLoan(GameState game[], int playerIndex)
217 {
218     int remainingRounds;
219     int cash;
220     int loanAmount;
221
222     remainingRounds = game[0].players[playerIndex].loan.remainingRounds;
223     cash = game[0].players[playerIndex].cash;
224     loanAmount = game[0].players[playerIndex].loan.amount;
225
226     switch(game[0].players[playerIndex].strategy)
227     {
228         case AGGRESSIVE_INVESTOR:
229             /* prefers to borrow more rather than extend */
230             return 0;
231
232         case CONSERVATIVE_BANKER:
233             return (remainingRounds <= 5 && cash < loanAmount);
234
235         case RISK_TAKER:
236             return (remainingRounds <= 10);
237
238         case OPPORTUNISTIC_TRADER:
239             return (remainingRounds <= 5);
240
241         default:
242             return 0;
243     }
244 }

```

```
245
246 /* Rule-LK 5: whether a player wants to refinance at the current rate
247 (only makes sense when the current economy rate is lower). */
248 int wantsRefinance(GameState game[], int playerIndex)
249 {
250     return (game[0].economy.loanInterestRate <
251             game[0].players[playerIndex].loan.interestRate);
252 }
253
254 /* Which insurance policy (if any) a player wants for a property. */
255 InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
256 {
257     int hasHotel;
258     int purchasePrice;
259
260     hasHotel = game[0].board[propIndex].property.hotel;
261     purchasePrice = game[0].board[propIndex].property.purchasePrice;
262
263     switch(game[0].players[playerIndex].strategy)
264     {
265         case AGGRESSIVE_INVESTOR:
266             if(hasHotel)
267                 return COMPREHENSIVE_INSURANCE;
268             return BASIC_INSURANCE;
269
270         case CONSERVATIVE_BANKER:
271             return COMPREHENSIVE_INSURANCE;
272
273         case RISK_TAKER:
274             /* only insures after already suffering a loss */
275             if(!game[0].players[playerIndex].sufferedLoss)
276                 return NO_INSURANCE;
277             if(hasHotel)
278                 return BUSINESS_INTERRUPTION;
279             return BASIC_INSURANCE;
280
281         case OPPORTUNISTIC_TRADER:
282             if(purchasePrice >= EXPENSIVE_PROP_PRICE)
283                 return COMPREHENSIVE_INSURANCE;
284             return NO_INSURANCE;
285
286         default:
287             return NO_INSURANCE;
288     }
289 }
290
291 /* Whether a player will pay to renovate away age depreciation. */
292 int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
293 {
294     switch(game[0].players[playerIndex].strategy)
295     {
296         case CONSERVATIVE_BANKER:
297             return (depreciationPercent >= 10);
298
299         case OPPORTUNISTIC_TRADER:
300             return (depreciationPercent >= 15);
301
302         default:
303             /* not specified for these two strategies - reasonable default */
304             return (depreciationPercent >= 20);
305     }
306 }
307
308 int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
309 {
310     int cash;
311
312     cash = game[0].players[playerIndex].cash;
313
314     if(game[0].players[playerIndex].strategy == RISK_TAKER)
315     {
316         /* ignores depreciation until repair becomes unavoidable */
317         return (condition < 25 && cash >= cost);
318     }
319
320     return (cash - cost >= cash * 20 / 100);
321 }
322
323 int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
324 {
325     int marketValue;
326 }
```

```

327     if(game[0].players[playerIndex].cash < candidateBid)
328         return 0;
329
330     marketValue = currentMarketValue(game, propIndex);
331
332     switch(game[0].players[playerIndex].strategy)
333     {
334         case AGGRESSIVE_INVESTOR:
335             return (candidateBid <= (marketValue * 120) / 100);
336
337         case CONSERVATIVE_BANKER:
338             return (candidateBid < marketValue);
339
340         case RISK_TAKER:
341             return 1;
342
343         case OPPORTUNISTIC_TRADER:
344             return (candidateBid <= (marketValue * 80) / 100);
345
346         default:
347             return 0;
348     }
349 }
350
351 int shouldMortgage(GameState game[], int playerIndex)
352 {
353     int cash;
354
355     cash = game[0].players[playerIndex].cash;
356
357     switch(game[0].players[playerIndex].strategy)
358     {
359         case CONSERVATIVE_BANKER:
360             return (cash < COMFORTABLE_CASH);
361
362         case RISK_TAKER:
363             return (cash < MIN_CASH_FLOOR);
364
365         case AGGRESSIVE_INVESTOR:
366             return (cash < MIN_CASH_SAFETY);
367
368         case OPPORTUNISTIC_TRADER:
369             return (cash < TRADER_CASH_FLOOR);
370
371         default:
372             return 0;
373     }
374 }
375
376 int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
377 {
378     return (game[0].players[playerIndex].cash - redeemCost >= redeemCost);
379 }
380
381 /* Whether a player pays bail after failing to roll doubles.
382    The assignment doesn't specify which strategy prefers which
383    option, so this is a reasonable assumption per strategy. */
384 int shouldPayBail(GameState game[], int playerIndex)
385 {
386     int cash;
387
388     cash = game[0].players[playerIndex].cash;
389
390     switch(game[0].players[playerIndex].strategy)
391     {
392         case CONSERVATIVE_BANKER:
393             return (cash >= BAIL_CASH_SPARE);
394
395         case RISK_TAKER:
396             return 0;
397
398         case AGGRESSIVE_INVESTOR:
399         case OPPORTUNISTIC_TRADER:
400             return (cash >= RICH_CASH_LINE);
401
402         default:
403             return 0;
404     }
405 }

```

finance.c

```

1  #include <stdio.h>
2  #include "types.h"
3  #include "functions.h"
4
5  /* Sets a square back to "unowned": no owner, no mortgage, no loan
6     pledge and no insurance. Used when selling, foreclosing or
7     liquidating a property. */
8  void stripOwnership(GameState game[], int index)
9  {
10     game[0].board[index].property.owner = -1;
11     game[0].board[index].property.mortgaged = 0;
12     game[0].board[index].property.loanLocked = 0;
13     game[0].board[index].property.insurance = NO_INSURANCE;
14 }
15
16 /* Clears a player's loan completely (amount, rate and rounds). */
17 void resetLoan(GameState game[], int playerIndex)
18 {
19     game[0].players[playerIndex].loan.active = 0;
20     game[0].players[playerIndex].loan.amount = 0;
21     game[0].players[playerIndex].loan.interestRate = 0;
22     game[0].players[playerIndex].loan.remainingRounds = 0;
23 }
24
25 /* Adds delta to the owned-count that matches the square's type
26     (property, railway or utility). Pass 1 when buying, -1 when
27     selling or losing a square. */
28 void adjustOwnedCount(GameState game[], int playerIndex, int index, int delta)
29 {
30     if(game[0].board[index].type == PROPERTY)
31         game[0].players[playerIndex].propertiesOwned += delta;
32     else if(game[0].board[index].type == RAILWAY)
33         game[0].players[playerIndex].railwaysOwned += delta;
34     else if(game[0].board[index].type == UTILITY)
35         game[0].players[playerIndex].utilitiesOwned += delta;
36 }
37
38 /* The colour group of a square, or -1 if it isn't a property. */
39 PropertyGroup groupOf(GameState game[], int index)
40 {
41     if(game[0].board[index].type == PROPERTY)
42         return game[0].board[index].property.group;
43
44     return (PropertyGroup)-1;
45 }
46
47 void receiveMoney(GameState game[], int playerIndex, int amount)
48 {
49     /* bankrupt players take no further part in the game (Section 1.3) */
50     if(game[0].players[playerIndex].bankrupt)
51         return;
52
53     game[0].players[playerIndex].cash += amount;
54 }
55
56 void payMoney(GameState game[], int playerIndex, int amount)
57 {
58     game[0].players[playerIndex].cash -= amount;
59
60     /* Section 3.4: a Risk Taker facing bankruptcy first sells their
61        cheapest undeveloped property to the Bank to raise cash.
62        Loan-locked collateral (Rule-LK 3) can't be sold, so stop as
63        soon as a sale fails to reduce the stock of undeveloped
64        properties - otherwise this loop would never end. */
65     while(game[0].players[playerIndex].cash < 0 &&
66           !game[0].players[playerIndex].bankrupt &&
67           game[0].players[playerIndex].strategy == RISK_TAKER)
68     {
69         int before = countUndevelopedProperties(game, playerIndex);
70
71         sellLowValueProperty(game, playerIndex);
72
73         if(countUndevelopedProperties(game, playerIndex) >= before)
74             break;
75     }
76
77     if(game[0].players[playerIndex].cash < 0 && !game[0].players[playerIndex].bankrupt)
78     {
79         game[0].players[playerIndex].bankrupt = 1;
80     }

```

```

81     printf("\n*** BANKRUPTCY ***\n");
82     printf("%s has been declared bankrupt.\n",
83           game[0].players[playerIndex].name);
84
85     liquidateBankruptAssets(game, playerIndex);
86
87     /* cash never stays negative - assets were liquidated, so any
88        remaining debt is written off (Section 1.3) */
89     game[0].players[playerIndex].cash = 0;
90 }
91 }
92
93 /* Rule 14: on bankruptcy, buildings are demolished and every
94    property/railway/utility owned is auctioned off (Rule-LK 19). */
95 void liquidateBankruptAssets(GameState game[], int playerIndex)
96 {
97     int i;
98
99     printf("%s's remaining assets are being liquidated.\n",
100          game[0].players[playerIndex].name);
101
102     resetLoan(game, playerIndex);
103
104     for(i = 0; i < BOARD_SIZE; i++)
105     {
106         if(game[0].board[i].property.owner != playerIndex)
107             continue;
108
109         demolishBuildingsOn(game, i);
110
111         stripOwnership(game, i);
112         game[0].board[i].property.damaged = 0;
113         game[0].board[i].property.repairCostOwed = 0;
114
115         adjustOwnedCount(game, playerIndex, i, -1);
116
117         runAuction(game, i, -1);
118     }
119 }
120
121 /* Income Tax (Rule-LK): 15% of the player's net worth at the base
122    rate. Market conditions scale the rate (currentTaxRatePercent). */
123 void payTax(GameState game[], int playerIndex)
124 {
125     int rate;
126     int netWorth;
127     int amount;
128
129     rate = currentTaxRatePercent(game, INCOME_TAX_RATE);
130     netWorth = calculateNetWorth(game, playerIndex);
131     amount = (netWorth * rate) / 100;
132
133     /* a bankrupt player's net worth can be negative - a tax can never
134        pay the player money, so clamp it at zero */
135     if(amount < 0)
136         amount = 0;
137
138     printf("\n%s landed on Income Tax.\n",
139          game[0].players[playerIndex].name);
140
141     printf("Tax Rate : %d%%\n", rate);
142     printf("Net Worth : LKR %d\n", netWorth);
143     printf("%s paid tax : LKR %d\n",
144          game[0].players[playerIndex].name,
145          amount);
146
147     payMoney(game, playerIndex, amount);
148 }
149
150 /* Community Development Fund (Rule-LK): 10% of the player's total
151    assets at current market rate. Only property (land) counts -
152    buildings are not included. The 10% rate is scaled by market
153    conditions just like the Income Tax rate. */
154 void payCommunityFundTax(GameState game[], int playerIndex)
155 {
156     int rate;
157     int assets;
158     int amount;
159
160     rate = currentTaxRatePercent(game, COMMUNITY_FUND_TAX_RATE);
161     assets = calculatePropertyValue(game, playerIndex);
162     amount = (assets * rate) / 100;

```

```
163
164     printf("\n%s landed on Community Development Fund.\n",
165            game[0].players[playerIndex].name);
166
167     printf("Tax Rate : %d%%\n", rate);
168     printf("Property Assets (market rate) : LKR %d\n", assets);
169     printf("%s paid community development tax : LKR %d\n",
170            game[0].players[playerIndex].name,
171            amount);
172
173     payMoney(game, playerIndex, amount);
174 }
175
176 int findPropertyToMortgage(GameState game[], int playerIndex)
177 {
178     int i;
179
180     for(i = 0; i < BOARD_SIZE; i++)
181     {
182         if(game[0].board[i].property.owner != playerIndex)
183             continue;
184
185         if(game[0].board[i].property.mortgaged)
186             continue;
187
188         if(game[0].board[i].property.loanLocked)
189             continue;
190
191         /* must be undeveloped before it can be mortgaged */
192         if(game[0].board[i].type == PROPERTY &&
193            (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
194         {
195             continue;
196         }
197
198         return i;
199     }
200
201     return -1;
202 }
203
204 int findMortgagedProperty(GameState game[], int playerIndex)
205 {
206     int i;
207
208     for(i = 0; i < BOARD_SIZE; i++)
209     {
210         if(game[0].board[i].property.owner == playerIndex &&
211            game[0].board[i].property.mortgaged)
212         {
213             return i;
214         }
215     }
216
217     return -1;
218 }
219
220 void mortgageProperty(GameState game[], int playerIndex)
221 {
222     int propIndex;
223     int payout;
224     PropertyGroup group;
225
226     if(!shouldMortgage(game, playerIndex))
227         return;
228
229     propIndex = findPropertyToMortgage(game, playerIndex);
230
231     if(propIndex == -1)
232         return;
233
234     /* Rule-LK 34: a booming group's mortgage values rise by 15%, a
235        declining group's fall by 10%. Railways and utilities are
236        unaffected. */
237     group = groupOf(game, propIndex);
238
239     payout = (game[0].board[propIndex].property.mortgageValue *
240              modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1)) / 100;
241
242     game[0].board[propIndex].property.mortgaged = 1;
243
244     receiveMoney(game, playerIndex, payout);
245 }
```

```

245
246     printf("\n%s mortgaged %s for LKR %d.\n",
247           game[0].players[playerIndex].name,
248           game[0].board[propIndex].name,
249           payout);
250 }
251
252 void redeemMortgage(GameState game[], int playerIndex)
253 {
254     int propIndex;
255     int redeemCost;
256
257     propIndex = findMortgagedProperty(game, playerIndex);
258
259     if(propIndex == -1)
260         return;
261
262     /* pay back the mortgage value plus 10% interest */
263     redeemCost = (game[0].board[propIndex].property.mortgageValue * MORTGAGE_REPAY_PERCENT) / 100;
264
265     if(!shouldRedeemMortgage(game, playerIndex, redeemCost))
266         return;
267
268     if(game[0].players[playerIndex].cash < redeemCost)
269         return;
270
271     payMoney(game, playerIndex, redeemCost);
272
273     game[0].board[propIndex].property.mortgaged = 0;
274
275     printf("\n%s redeemed the mortgage on %s for LKR %d.\n",
276           game[0].players[playerIndex].name,
277           game[0].board[propIndex].name,
278           redeemCost);
279 }
280
281 void handleMortgageDecisions(GameState game[], int playerIndex)
282 {
283     redeemMortgage(game, playerIndex);
284     mortgageProperty(game, playerIndex);
285 }
286
287 /* Rent for a normal property, based on houses/hotel (Table 6),
288    scaled by any active bonuses and by building condition (Table 3). */
289 int calculateRent(GameState game[], int position)
290 {
291     int rent;
292     int conditionPercent;
293     int houses;
294     int hotel;
295     int baseRent;
296     PropertyGroup group;
297     int condition;
298
299     houses = game[0].board[position].property.houses;
300     hotel = game[0].board[position].property.hotel;
301     baseRent = game[0].board[position].property.baseRent;
302     group = game[0].board[position].property.group;
303     condition = game[0].board[position].property.condition;
304
305     if(hotel)
306     {
307         rent = baseRent * 10;
308     }
309     else
310     {
311         if(houses == 0)
312             rent = baseRent * 1;
313         else if(houses == 1)
314             rent = baseRent * 2;
315         else if(houses == 2)
316             rent = baseRent * 3;
317         else if(houses == 3)
318             rent = baseRent * 5;
319         else
320             rent = baseRent * 7;
321     }
322
323     /* Rule-LK 34: all matching modifiers multiply together. */
324     rent = (rent * modifierMultiplier(game, MOD_GROUP_RENT, group, -1)) / 100;
325     rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;
326

```

```
327     if(hotel)
328         rent = (rent * modifierMultiplier(game, MOD_HOTEL_RENT, -1, -1)) / 100;
329
330     if(houses > 0 || hotel)
331     {
332         conditionPercent = rentConditionPercent(condition);
333         rent = (rent * conditionPercent) / 100;
334     }
335
336     return rent;
337 }
338
339 /* Rent for a railway, based on how many the same owner has (Table 7). */
340 int calculateRailwayRent(GameState game[], int playerIndex)
341 {
342     int i;
343     int count;
344     int baseRent;
345
346     count = 0;
347
348     for(i = 0; i < BOARD_SIZE; i++)
349     {
350         if(game[0].board[i].type == RAILWAY &&
351            game[0].board[i].property.owner == playerIndex)
352         {
353             count++;
354         }
355     }
356
357     if(count == 1)
358         baseRent = RAILWAY_RENT_1;
359     else if(count == 2)
360         baseRent = RAILWAY_RENT_2;
361     else if(count == 3)
362         baseRent = RAILWAY_RENT_3;
363     else if(count == 4)
364         baseRent = RAILWAY_RENT_4;
365     else
366         baseRent = 0;
367
368     baseRent = (baseRent * modifierMultiplier(game, MOD_RAIL_RENT, -1, -1)) / 100;
369     baseRent = (baseRent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;
370
371     return baseRent;
372 }
373
374 /* Rent for a utility, based on the dice roll just made (Table 8). */
375 int calculateUtilityRent(GameState game[], int playerIndex, int diceValue)
376 {
377     int i;
378     int count;
379     int rent;
380
381     count = 0;
382
383     for(i = 0; i < BOARD_SIZE; i++)
384     {
385         if(game[0].board[i].type == UTILITY &&
386            game[0].board[i].property.owner == playerIndex)
387         {
388             count++;
389         }
390     }
391
392     if(count == 2)
393         rent = UTILITY_RENT_TWO * diceValue;
394     else if(count == 1)
395         rent = UTILITY_RENT_ONE * diceValue;
396     else
397         rent = 0;
398
399     rent = (rent * modifierMultiplier(game, MOD_UTIL_RENT, -1, -1)) / 100;
400     rent = (rent * modifierMultiplier(game, MOD_RENT_GLOBAL, -1, -1)) / 100;
401
402     return rent;
403 }
404
405 int groupSize(GameState game[], PropertyGroup group)
406 {
407     int i;
408     int count;
```

```
409
410     count = 0;
411
412     for(i = 0; i < BOARD_SIZE; i++)
413     {
414         if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
415             count++;
416     }
417
418     return count;
419 }
420
421 int ownsMonopoly(GameState game[], int playerIndex, PropertyGroup group)
422 {
423     int i;
424     int owned;
425
426     owned = 0;
427
428     for(i = 0; i < BOARD_SIZE; i++)
429     {
430         if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
431         {
432             if(game[0].board[i].property.owner == playerIndex)
433                 owned++;
434         }
435     }
436
437     return (owned == groupSize(game, group));
438 }
439
440 /* Builds one house (or upgrades to a hotel) somewhere in this
441 colour group. Always adds to whichever property currently has the
442 fewest houses, so development stays even (Rule 9). Returns 1 if
443 something was actually built, 0 if not - constructBuildings()
444 uses this to keep building repeatedly in the same turn. */
445 int developGroup(GameState game[], int playerIndex, PropertyGroup group)
446 {
447     int i;
448     int minHouses;
449     int targetIndex;
450     int allFourHouses;
451
452     if(!ownsMonopoly(game, playerIndex, group))
453         return 0;
454
455     minHouses = MAX_HOUSES + 1;
456     targetIndex = -1;
457     allFourHouses = 1;
458
459     for(i = 0; i < BOARD_SIZE; i++)
460     {
461         if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
462         {
463             /* can't build on a mortgaged, damaged or loan-locked
464             property - those earn no rent and/or are pledged */
465             if(game[0].board[i].property.mortgaged ||
466                game[0].board[i].property.damaged ||
467                game[0].board[i].property.loanLocked)
468             {
469                 /* Rule 9: a blocked member that isn't fully developed
470                 means the group can't support a hotel yet */
471                 if(!game[0].board[i].property.hotel &&
472                    game[0].board[i].property.houses < MAX_HOUSES)
473                 {
474                     allFourHouses = 0;
475                 }
476
477                 continue;
478             }
479
480             if(game[0].board[i].property.hotel)
481                 continue;
482
483             if(game[0].board[i].property.houses < MAX_HOUSES)
484                 allFourHouses = 0;
485
486             if(game[0].board[i].property.houses < minHouses)
487             {
488                 minHouses = game[0].board[i].property.houses;
489                 targetIndex = i;
490             }
491         }
492     }
```

```

491     }
492 }
493
494 if(targetIndex == -1)
495     return 0; /* every property already has a hotel */
496
497 /* Rule 9: never pile a 5th house onto one property just because a
498    group member is blocked from building */
499 if(game[0].board[targetIndex].property.houses >= MAX_HOUSES)
500     return 0;
501
502 if(allFourHouses)
503 {
504     int hotelCost;
505     char costBuf[32];
506
507     /* Rule-LK 13/31/34: base hotel cost is permanent; temporary
508        events only add modifiers on top. A market boom's +10%
509        applies only to the booming group, while event/regulation
510        modifiers (group -1) are global. */
511     hotelCost = (game[0].board[targetIndex].property.hotelCost *
512                 modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;
513
514     if(!shouldConstruct(game, playerIndex, hotelCost, 1))
515         return 0;
516
517     if(game[0].players[playerIndex].cash < hotelCost)
518         return 0;
519
520     payMoney(game, playerIndex, hotelCost);
521
522     game[0].board[targetIndex].property.houses = 0;
523     game[0].board[targetIndex].property.hotel = 1;
524
525     printf("\n%s upgraded %s to a Hotel.\n",
526           game[0].players[playerIndex].name,
527           game[0].board[targetIndex].name);
528
529     formatLKR(hotelCost, costBuf);
530     printf("Construction Cost : LKR %s.\n", costBuf);
531
532     return 1;
533 }
534
535 {
536     int houseCost;
537     char costBuf[32];
538
539     houseCost = (game[0].board[targetIndex].property.houseCost *
540                 modifierMultiplier(game, MOD_CONSTRUCTION, group, -1)) / 100;
541
542     if(!shouldConstruct(game, playerIndex, houseCost, 0))
543         return 0;
544
545     if(game[0].players[playerIndex].cash < houseCost)
546         return 0;
547
548     payMoney(game, playerIndex, houseCost);
549
550     game[0].board[targetIndex].property.houses++;
551
552     printf("\n%s constructed one house on %s.\n",
553           game[0].players[playerIndex].name,
554           game[0].board[targetIndex].name);
555
556     formatLKR(houseCost, costBuf);
557     printf("Construction Cost : LKR %s.\n", costBuf);
558
559     return 1;
560 }
561 }
562
563 /* Keeps building on each colour group, one step at a time, until
564    either nothing more can be built there this turn (unaffordable,
565    the strategy doesn't want to, or the group is fully built up) -
566    this is what lets a strategy build "the maximum possible number
567    of houses immediately" in a single turn instead of one per turn. */
568 void constructBuildings(GameState game[], int playerIndex)
569 {
570     PropertyGroup group;
571     int builtSomething;
572     int safetyLimit;

```

```
573
574     if(game[0].economy.constructionSuspendedRoundsLeft > 0)
575         return;
576
577     for(group = BROWN; group < NO_GROUP; group++)
578     {
579         safetyLimit = 0;
580
581         do
582         {
583             builtSomething = developGroup(game, playerIndex, group);
584             safetyLimit++;
585         }
586         while(builtSomething && safetyLimit < 20);
587     }
588 }
589
590 int calculatePropertyValue(GameState game[], int playerIndex)
591 {
592     int i;
593     int total;
594
595     total = 0;
596
597     for(i = 0; i < BOARD_SIZE; i++)
598     {
599         if(game[0].board[i].property.owner != playerIndex)
600             continue;
601
602         /* mortgaged properties contribute zero to net worth */
603         if(game[0].board[i].property.mortgaged)
604             continue;
605
606         /* railways and utilities also use currentMarketValue so their
607            value modifiers (rail/port/water effects) apply too */
608         total += currentMarketValue(game, i);
609     }
610
611     return total;
612 }
613
614 int calculateBuildingValue(GameState game[], int playerIndex)
615 {
616     int i;
617     int total;
618
619     total = 0;
620
621     for(i = 0; i < BOARD_SIZE; i++)
622     {
623         if(game[0].board[i].type != PROPERTY)
624             continue;
625
626         if(game[0].board[i].property.owner != playerIndex)
627             continue;
628
629         if(game[0].board[i].property.hotel)
630             total += game[0].board[i].property.hotelCost;
631         else
632             total += game[0].board[i].property.houses * game[0].board[i].property.houseCost;
633     }
634
635     return total;
636 }
637
638 /* Rule 15 net worth formula: cash + property value + building value,
639    minus any outstanding loan amount. */
640 int calculateNetWorth(GameState game[], int playerIndex)
641 {
642     int cash;
643     int propertyValue;
644     int buildingValue;
645     int outstandingLoans;
646
647     cash = game[0].players[playerIndex].cash;
648     propertyValue = calculatePropertyValue(game, playerIndex);
649     buildingValue = calculateBuildingValue(game, playerIndex);
650
651     outstandingLoans = 0;
652
653     if(game[0].players[playerIndex].loan.active)
654         outstandingLoans = game[0].players[playerIndex].loan.amount;
```

```
655
656     return cash + propertyValue + buildingValue - outstandingLoans;
657 }
658
659 int countUndevelopedProperties(GameState game[], int playerIndex)
660 {
661     int i;
662     int count;
663
664     count = 0;
665
666     for(i = 0; i < BOARD_SIZE; i++)
667     {
668         if(game[0].board[i].type == PROPERTY &&
669            game[0].board[i].property.owner == playerIndex &&
670            game[0].board[i].property.houses == 0 &&
671            !game[0].board[i].property.hotel)
672         {
673             count++;
674         }
675     }
676
677     return count;
678 }
679
680 /* True if buying the property at pos would complete the whole colour
681    group for the player (Section 3.1). */
682 int wouldCompleteMonopoly(GameState game[], int playerIndex, int pos)
683 {
684     int i;
685     PropertyGroup group;
686
687     if(game[0].board[pos].type != PROPERTY)
688         return 0;
689
690     if(game[0].board[pos].property.owner != -1)
691         return 0;
692
693     group = game[0].board[pos].property.group;
694
695     for(i = 0; i < BOARD_SIZE; i++)
696     {
697         if(game[0].board[i].type == PROPERTY && game[0].board[i].property.group == group)
698         {
699             if(i != pos && game[0].board[i].property.owner != playerIndex)
700                 return 0;
701         }
702     }
703
704     return 1;
705 }
706
707 /* Sells the player's lowest-value undeveloped property back to the
708    Bank at market value (Section 3.4 bankruptcy tactic). */
709 void sellLowValueProperty(GameState game[], int playerIndex)
710 {
711     int i;
712     int best;
713     int bestValue;
714     int value;
715
716     best = -1;
717     bestValue = 0;
718
719     for(i = 0; i < BOARD_SIZE; i++)
720     {
721         if(game[0].board[i].type != PROPERTY)
722             continue;
723
724         if(game[0].board[i].property.owner != playerIndex)
725             continue;
726
727         if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
728             continue;
729
730         /* Rule-LK 3: loan-locked collateral can never be sold */
731         if(game[0].board[i].property.loanLocked)
732             continue;
733
734         value = currentMarketValue(game, i);
735
736         if(best == -1 || value < bestValue)
```

```
737     {
738         best = i;
739         bestValue = value;
740     }
741 }
742
743 if(best == -1)
744     return;
745
746 receiveMoney(game, playerIndex, bestValue);
747
748 stripOwnership(game, best);
749
750 adjustOwnedCount(game, playerIndex, best, -1);
751
752 printf("\n%s sold %s to the Bank for LKR %d.\n",
753        game[0].players[playerIndex].name,
754        game[0].board[best].name,
755        bestValue);
756 }
757
758 /* Section 3.4: the Opportunistic Trader sells properties expected to
759    lose value following economic events - any undeveloped property in
760    a group that is currently declining, under a global downturn, or
761    carrying its own negative value modifier. Loan-locked collateral
762    (Rule-LK 3) is never sold. */
763 void sellDecliningProperties(GameState game[], int playerIndex)
764 {
765     int i;
766     int globalMult;
767
768     if(game[0].players[playerIndex].strategy != OPPORTUNISTIC_TRADER)
769         return;
770
771     if(game[0].players[playerIndex].bankrupt)
772         return;
773
774     globalMult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);
775
776     for(i = 0; i < BOARD_SIZE; i++)
777     {
778         int groupMult;
779         int indexMult;
780         int value;
781
782         if(game[0].board[i].type != PROPERTY)
783             continue;
784
785         if(game[0].board[i].property.owner != playerIndex)
786             continue;
787
788         if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
789             continue;
790
791         if(game[0].board[i].property.loanLocked)
792             continue;
793
794         groupMult = modifierMultiplier(game, MOD_GROUP_VALUE,
795                                       game[0].board[i].property.group, -1);
796         indexMult = modifierMultiplier(game, MOD_INDEX_VALUE, -1, i);
797
798         if(globalMult >= 100 && groupMult >= 100 && indexMult >= 100)
799             continue;
800
801         value = currentMarketValue(game, i);
802
803         receiveMoney(game, playerIndex, value);
804
805         stripOwnership(game, i);
806
807         adjustOwnedCount(game, playerIndex, i, -1);
808
809         printf("\n%s sold %s to the Bank for LKR %d "
810                "(expected to lose value).\n",
811                game[0].players[playerIndex].name,
812                game[0].board[i].name,
813                value);
814     }
815 }
816
817 /* Releases the player's lowest-value undeveloped property and puts
818    it up for auction (Anti-Speculation Act enforcement). */
```

```

819 void sellUndevelopedPropertyToAuction(GameState game[], int playerIndex)
820 {
821     int i;
822     int best;
823     int bestValue;
824     int value;
825
826     best = -1;
827     bestValue = 0;
828
829     for(i = 0; i < BOARD_SIZE; i++)
830     {
831         if(game[0].board[i].type != PROPERTY)
832             continue;
833
834         if(game[0].board[i].property.owner != playerIndex)
835             continue;
836
837         if(game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel)
838             continue;
839
840         /* Rule-LK 3: loan-locked collateral can never be auctioned */
841         if(game[0].board[i].property.loanLocked)
842             continue;
843
844         value = currentMarketValue(game, i);
845
846         if(best == -1 || value < bestValue)
847         {
848             best = i;
849             bestValue = value;
850         }
851     }
852
853     if(best == -1)
854         return;
855
856     printf("\nAnti-Speculation Act : %s must sell %s.\n",
857         game[0].players[playerIndex].name,
858         game[0].board[best].name);
859
860     stripOwnership(game, best);
861
862     adjustOwnedCount(game, playerIndex, best, -1);
863
864     runAuction(game, best, playerIndex);
865 }
866
867 /* Sells excess undeveloped properties until the player is back at
868    the allowed limit (or nothing more can be sold). */
869 void forceSellExcess(GameState game[], int playerIndex)
870 {
871     int excess;
872
873     excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
874
875     while(excess > 0)
876     {
877         int before = countUndevelopedProperties(game, playerIndex);
878
879         sellUndevelopedPropertyToAuction(game, playerIndex);
880
881         /* nothing sellable left (e.g. every excess property is
882            loan-locked) - stop to avoid a non-terminating loop */
883         if(countUndevelopedProperties(game, playerIndex) >= before)
884             break;
885
886         excess = countUndevelopedProperties(game, playerIndex) - MAX_UNDEVELOPED_PROPS;
887     }
888 }
889
890 /* Rule-LK 8 (Anti-Speculation Act): once active, a player may keep
891    at most three undeveloped properties. After 5 consecutive rounds
892    above the limit the excess is force-auctioned. */
893 void enforceAntiSpeculation(GameState game[])
894 {
895     int i;
896     int undeveloped;
897
898     if(!game[0].economy.antiSpeculationActive)
899     {
900         for(i = 0; i < MAX_PLAYERS; i++)

```

```

901         game[0].players[i].antiSpecRounds = 0;
902
903     return;
904 }
905
906 for(i = 0; i < MAX_PLAYERS; i++)
907 {
908     if(game[0].players[i].bankrupt)
909     {
910         game[0].players[i].antiSpecRounds = 0;
911         continue;
912     }
913
914     undeveloped = countUndevelopedProperties(game, i);
915
916     if(undeveloped > MAX_UNDEVELOPED_PROPS)
917     {
918         game[0].players[i].antiSpecRounds++;
919
920         if(game[0].players[i].antiSpecRounds >= ANTI_SPEC_TRIGGER_ROUNDS)
921         {
922             forceSellExcess(game, i);
923             game[0].players[i].antiSpecRounds = 0;
924         }
925     }
926     else
927     {
928         game[0].players[i].antiSpecRounds = 0;
929     }
930 }
931 }
932
933 void buyProperty(GameState game[], int playerIndex)
934 {
935     int pos;
936     int price;
937     int wantsToBuy;
938     char priceBuf[32];
939     char balanceBuf[32];
940
941     pos = game[0].players[playerIndex].position;
942
943     if(game[0].board[pos].type != PROPERTY &&
944        game[0].board[pos].type != RAILWAY &&
945        game[0].board[pos].type != UTILITY)
946     {
947         return;
948     }
949
950     if(game[0].board[pos].property.owner != -1)
951         return; /* already owned */
952
953     price = currentMarketValue(game, pos);
954
955     if(game[0].board[pos].type == PROPERTY)
956     {
957         /* Rule-LK 31/34: a booming group's direct purchase prices
958            rise by 15% (group-scoped, not global). Railways and
959            utilities are priced at their current market value, so
960            their value modifiers (MOD_RAIL_VALUE, MOD_INDEX_VALUE)
961            affect their purchase price too. */
962         price = (price *
963                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
964                                     game[0].board[pos].property.group, -1)) / 100;
965     }
966
967     wantsToBuy = shouldBuyProperty(game, playerIndex);
968
969     if(game[0].players[playerIndex].cash < price)
970         wantsToBuy = 0;
971
972     /* Rule 5: if they don't buy directly, it goes to auction */
973     if(!wantsToBuy)
974     {
975         runAuction(game, pos, -1);
976         return;
977     }
978
979     payMoney(game, playerIndex, price);
980
981     game[0].board[pos].property.owner = playerIndex;
982

```

```

983     if(game[0].board[pos].type == PROPERTY)
984         game[0].players[playerIndex].propertiesOwned++;
985     else if(game[0].board[pos].type == RAILWAY)
986         game[0].players[playerIndex].railwaysOwned++;
987     else if(game[0].board[pos].type == UTILITY)
988         game[0].players[playerIndex].utilitiesOwned++;
989
990     formatLKR(price, priceBuf);
991     formatLKR(game[0].players[playerIndex].cash, balanceBuf);
992
993     printf("\n%s purchased %s for LKR %s.\n",
994           game[0].players[playerIndex].name,
995           game[0].board[pos].name,
996           priceBuf);
997
998     printf("Remaining Balance : LKR %s.\n",
999           balanceBuf);
1000 }
1001
1002 void payRent(GameState game[], int playerIndex, int diceValue)
1003 {
1004     int pos;
1005     int owner;
1006     int rent;
1007     char rentBuf[32];
1008
1009     pos = game[0].players[playerIndex].position;
1010
1011     if(game[0].board[pos].type != PROPERTY &&
1012        game[0].board[pos].type != RAILWAY &&
1013        game[0].board[pos].type != UTILITY)
1014     {
1015         return;
1016     }
1017
1018     owner = game[0].board[pos].property.owner;
1019
1020     if(owner == playerIndex)
1021     {
1022         /* landed on your own square - no rent, maybe renovate */
1023         if(game[0].board[pos].type == PROPERTY)
1024             tryRenovateAgeDepreciation(game, playerIndex, pos);
1025         return;
1026     }
1027
1028     if(owner == -1)
1029         return;
1030
1031     if(game[0].board[pos].property.mortgaged)
1032         return;
1033
1034     if(pos == game[0].economy.closedPropertyIndex && game[0].economy.closedPropertyRoundsLeft > 0)
1035     {
1036         printf("\n%s landed on %s, but it is closed (Political Rally) - no rent.\n",
1037               game[0].players[playerIndex].name, game[0].board[pos].name);
1038         return;
1039     }
1040
1041     if(game[0].board[pos].type == PROPERTY && game[0].board[pos].property.damaged)
1042     {
1043         printf("\n%s landed on %s, but it is damaged and collects no rent.\n",
1044               game[0].players[playerIndex].name, game[0].board[pos].name);
1045         return;
1046     }
1047
1048     if(game[0].board[pos].type == PROPERTY &&
1049        game[0].board[pos].property.lostIncomeRoundsLeft > 0)
1050     {
1051         printf("\n%s landed on %s, but it earns no rent right now "
1052               "(Business Interruption).\n",
1053               game[0].players[playerIndex].name, game[0].board[pos].name);
1054         return;
1055     }
1056
1057     if(game[0].board[pos].type == PROPERTY)
1058         rent = calculateRent(game, pos);
1059     else if(game[0].board[pos].type == RAILWAY)
1060         rent = calculateRailwayRent(game, owner);
1061     else
1062         rent = calculateUtilityRent(game, owner, diceValue);
1063
1064     payMoney(game, playerIndex, rent);

```

```
1065     receiveMoney(game, owner, rent);
1066
1067     formatLKR(rent, rentBuf);
1068
1069     printf("\n%s landed on %s.\n",
1070           game[0].players[playerIndex].name,
1071           game[0].board[pos].name);
1072
1073     printf("Rent Paid : LKR %s.\n", rentBuf);
1074
1075     printf("Owner : %s.\n", game[0].players[owner].name);
1076 }
```

game.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #include "types.h"
6  #include "functions.h"
7
8  int rollDice(void)
9  {
10     int die1 = rand() % 6 + 1;
11     int die2 = rand() % 6 + 1;
12
13     return die1 + die2;
14 }
15
16 /* Moves a player, returns 1 if they passed or landed on GO. */
17 int movePlayer(GameState game[], int playerIndex, int dice)
18 {
19     int oldPosition;
20     int passedGo;
21     char moneyBuffer[32];
22
23     oldPosition = game[0].players[playerIndex].position;
24     game[0].players[playerIndex].position = (oldPosition + dice) % BOARD_SIZE;
25
26     printf("%s moves from Square %d to Square %d.\n",
27           game[0].players[playerIndex].name,
28           oldPosition,
29           game[0].players[playerIndex].position);
30
31     passedGo = 0;
32
33     if(oldPosition + dice >= BOARD_SIZE)
34     {
35         passedGo = 1;
36
37         receiveMoney(game, playerIndex, GO_MONEY);
38
39         printf("%s passed GO.\n", game[0].players[playerIndex].name);
40
41         formatLKR(GO_MONEY, moneyBuffer);
42
43         printf("Collected LKR %s\n", moneyBuffer);
44
45         printf("Current Balance : LKR %d\n",
46               game[0].players[playerIndex].cash);
47     }
48
49     return passedGo;
50 }
51
52 /* Handles a player's turn while they're in jail.
53    Return codes:
54     0 = wasn't in jail - take a normal turn
55     1 = was in jail (released or not) - turn ends, no move this turn */
56 int handleJail(GameState game[], int playerIndex)
57 {
58     int die1, die2;
59
60     if(!game[0].players[playerIndex].inJail)
61         return 0;
62
63     die1 = rand() % 6 + 1;
64     die2 = rand() % 6 + 1;
65
66     printf("%s is in Jail. Rolled %d and %d.\n",
67           game[0].players[playerIndex].name, die1, die2);
68
69     if(die1 == die2)
70     {
71         printf("Doubles! %s is released from Jail.\n",
72               game[0].players[playerIndex].name);
73
74         /* Rule 13: rolling doubles releases the player, but they do
75            not move this turn - they remain on the Jail square and
76            move on their next turn. */
77         game[0].players[playerIndex].inJail = 0;
78         game[0].players[playerIndex].jailTurns = 0;
79
80         return 1;
```

```

81     }
82
83     /* Rule 13: paying bail is a strategy choice made after a failed
84        doubles roll. The player is released but does not move this
85        turn. */
86     if(shouldPayBail(game, playerIndex) &&
87        game[0].players[playerIndex].cash >= JAIL_BAIL)
88     {
89         printf("%s pays bail of LKR %d and is released from Jail.\n",
90             game[0].players[playerIndex].name, JAIL_BAIL);
91
92         payMoney(game, playerIndex, JAIL_BAIL);
93
94         game[0].players[playerIndex].inJail = 0;
95         game[0].players[playerIndex].jailTurns = 0;
96
97         return 1;
98     }
99
100    game[0].players[playerIndex].jailTurns++;
101
102    if(game[0].players[playerIndex].jailTurns >= JAIL_MAX_TURNS)
103    {
104        printf("%s has been in Jail for %d turns - must pay bail of "
105            "LKR %d and is released.\n",
106            game[0].players[playerIndex].name, JAIL_MAX_TURNS, JAIL_BAIL);
107
108        payMoney(game, playerIndex, JAIL_BAIL);
109
110        game[0].players[playerIndex].inJail = 0;
111        game[0].players[playerIndex].jailTurns = 0;
112
113        return 1;
114    }
115
116    printf("%s remains in Jail (%d/%d turns).\n",
117        game[0].players[playerIndex].name,
118        game[0].players[playerIndex].jailTurns,
119        JAIL_MAX_TURNS);
120
121    return 1;
122 }
123
124 /* One player's whole turn, matching Rule 3. Returns 1 if GO was
125    passed at any point during the turn. */
126 int playTurn(GameState game[], int playerIndex)
127 {
128     int dice;
129     int pos;
130     int passedGo;
131     int jailResult;
132     SquareType squareType;
133
134     if(game[0].players[playerIndex].bankrupt)
135         return 0;
136
137     printf("\n-----\n");
138     printf("%s's Turn\n", game[0].players[playerIndex].name);
139
140     /* Step 1: fix any damaged, worn, or neglected buildings first */
141     tryAutoRepair(game, playerIndex);
142     performMaintenance(game, playerIndex);
143     renovateStructuralDamage(game, playerIndex);
144
145     passedGo = 0;
146
147     jailResult = handleJail(game, playerIndex);
148
149     if(jailResult == 1)
150         return 0; /* was in jail - released or not, no move this turn */
151
152     /* jailResult == 0: wasn't in jail - take a completely normal turn. */
153     dice = rollDice();
154     printf("%s rolled %d.\n", game[0].players[playerIndex].name, dice);
155
156     if(movePlayer(game, playerIndex, dice))
157         passedGo = 1;
158
159     /* Step 4: resolve landing action */
160     pos = game[0].players[playerIndex].position;
161     squareType = game[0].board[pos].type;
162

```

```

163     if(squareType == PROPERTY || squareType == RAILWAY || squareType == UTILITY)
164     {
165         payRent(game, playerIndex, dice);
166         buyProperty(game, playerIndex);
167     }
168     else if(squareType == EVENT)
169     {
170         executeEvent(game, playerIndex);
171     }
172     else if(squareType == TAX)
173     {
174         payTax(game, playerIndex);
175     }
176     else if(squareType == COMMUNITY_FUND)
177     {
178         payCommunityFundTax(game, playerIndex);
179     }
180     else if(squareType == GO_TO_JAIL)
181     {
182         printf("%s is sent to Jail!\n", game[0].players[playerIndex].name);
183         game[0].players[playerIndex].position = JAIL_SQUARE;
184         game[0].players[playerIndex].inJail = 1;
185         game[0].players[playerIndex].jailTurns = 0;
186     }
187     else if(squareType == BANK)
188     {
189         handleBankVisit(game, playerIndex);
190     }
191     else if(squareType == INSURANCE)
192     {
193         handleInsuranceVisit(game, playerIndex);
194     }
195     /* GO, JAIL (just visiting), FREE_PARKING need no action */
196
197     /* Step 6: build if eligible */
198     constructBuildings(game, playerIndex);
199
200     /* Section 3.4: the Opportunistic Trader first dumps any property
201        that economic events have marked for a decline. */
202     sellDecliningProperties(game, playerIndex);
203
204     /* Step 7: mortgage or redeem if the player's situation calls for it */
205     handleMortgageDecisions(game, playerIndex);
206
207     return passedGo;
208 }
209
210 /* Resolves one group of (possibly tied) players into a strict order.
211 Writes the result into output[] starting at startIndex, and
212 returns how many players it wrote. Any players still tied after
213 rolling call this function again on just themselves. */
214 int resolveGroup(GameState game[], int groupPlayers[], int groupSize,
215                 int output[], int startIndex)
216 {
217     int rolls[MAX_PLAYERS];
218     int i, j, temp;
219     int runStart, runLength;
220     int tiedPlayers[MAX_PLAYERS];
221     int written;
222     int k;
223
224     written = 0;
225
226     for(i = 0; i < groupSize; i++)
227     {
228         rolls[i] = rollDice();
229         printf("%s rolls %d.\n", game[0].players[groupPlayers[i]].name, rolls[i]);
230     }
231
232     /* sort this group by roll, highest first */
233     for(i = 0; i < groupSize - 1; i++)
234     {
235         for(j = 0; j < groupSize - 1 - i; j++)
236         {
237             if(rolls[j] < rolls[j + 1])
238             {
239                 temp = rolls[j];
240                 rolls[j] = rolls[j + 1];
241                 rolls[j + 1] = temp;
242
243                 temp = groupPlayers[j];
244                 groupPlayers[j] = groupPlayers[j + 1];

```

```

245         groupPlayers[j + 1] = temp;
246     }
247 }
248 }
249
250 /* walk the sorted list - a run of equal rolls is a tie */
251 i = 0;
252
253 while(i < groupSize)
254 {
255     runStart = i;
256     runLength = 1;
257
258     while(runStart + runLength < groupSize &&
259           rolls[runStart + runLength] == rolls[runStart])
260     {
261         runLength++;
262     }
263
264     if(runLength == 1)
265     {
266         output[startIndex + written] = groupPlayers[runStart];
267         written++;
268     }
269     else
270     {
271         printf("Tie detected between %d players at %d. Rolling again.\n",
272               runLength, rolls[runStart]);
273
274         for(k = 0; k < runLength; k++)
275             tiedPlayers[k] = groupPlayers[runStart + k];
276
277         written += resolveGroup(game, tiedPlayers, runLength,
278                                output, startIndex + written);
279     }
280
281     i = runStart + runLength;
282 }
283
284 return written;
285 }
286
287 void determineTurnOrder(GameState game[], int turnOrder[])
288 {
289     int allPlayers[MAX_PLAYERS];
290     int i;
291
292     for(i = 0; i < MAX_PLAYERS; i++)
293         allPlayers[i] = i;
294
295     printf("\nDetermining the First Player\n");
296
297     resolveGroup(game, allPlayers, MAX_PLAYERS, turnOrder, 0);
298
299     printf("\n%s will begin the game.\n", game[0].players[turnOrder[0]].name);
300
301     printf("\nTurn order:\n");
302     for(i = 0; i < MAX_PLAYERS; i++)
303         printf("%s\n", game[0].players[turnOrder[i]].name);
304 }
305
306 int countHouses(GameState game[], int playerIndex)
307 {
308     int i;
309     int count;
310
311     count = 0;
312
313     for(i = 0; i < BOARD_SIZE; i++)
314     {
315         if(game[0].board[i].type == PROPERTY &&
316            game[0].board[i].property.owner == playerIndex &&
317            !game[0].board[i].property.hotel)
318         {
319             count += game[0].board[i].property.houses;
320         }
321     }
322
323     return count;
324 }
325
326 int countHotels(GameState game[], int playerIndex)

```

```

327 {
328     int i;
329     int count;
330
331     count = 0;
332
333     for(i = 0; i < BOARD_SIZE; i++)
334     {
335         if(game[0].board[i].type == PROPERTY &&
336            game[0].board[i].property.owner == playerIndex &&
337            game[0].board[i].property.hotel)
338         {
339             count++;
340         }
341     }
342
343     return count;
344 }
345
346 void displayRoundSummary(GameState game[], int round)
347 {
348     int i;
349
350     printf("\n=====\\n");
351     printf("Round %d Summary\\n", round);
352     printf("=====\\n");
353
354     for(i = 0; i < MAX_PLAYERS; i++)
355     {
356         char cashBuf[32];
357         char netBuf[32];
358         char loanBuf[32];
359
360         formatLKR(game[0].players[i].cash, cashBuf);
361         formatLKR(calculateNetWorth(game, i), netBuf);
362
363         printf("%s\\n", game[0].players[i].name);
364         printf("Cash : LKR %s\\n", cashBuf);
365         printf("Net Worth : LKR %s\\n", netBuf);
366         printf("Properties : %d\\n", game[0].players[i].propertiesOwned);
367         printf("Railways : %d\\n", game[0].players[i].railwaysOwned);
368         printf("Utilities : %d\\n", game[0].players[i].utilitiesOwned);
369         printf("Houses : %d\\n", countHouses(game, i));
370         printf("Hotels : %d\\n", countHotels(game, i));
371
372         if(game[0].players[i].loan.active)
373         {
374             formatLKR(game[0].players[i].loan.amount, loanBuf);
375             printf("Outstanding Loan : LKR %s\\n", loanBuf);
376         }
377         else
378         {
379             printf("Outstanding Loan : None\\n");
380         }
381
382         if(game[0].players[i].bankrupt)
383             printf("Status : BANKRUPT\\n");
384
385         printf("-----\\n");
386     }
387 }
388
389 int countSolventPlayers(GameState game[])
390 {
391     int i;
392     int count;
393
394     count = 0;
395
396     for(i = 0; i < MAX_PLAYERS; i++)
397     {
398         if(!game[0].players[i].bankrupt)
399             count++;
400     }
401
402     return count;
403 }
404
405 /* A round is not "each player took one turn" - it only finishes
406    once every player still in the game has passed GO at least once.
407    Since players move different distances each turn, this usually
408    takes several turns per player, not one - so this loop keeps

```

```
409     cycling through turnOrder until everyone has crossed GO. */
410 void playGame(GameState game[], int turnOrder[])
411 {
412     int round;
413     int turnPointer;
414     int playerIndex;
415     int passedGoThisRound[MAX_PLAYERS];
416     int i;
417     int allPassed;
418     int passedGo;
419
420     round = 1;
421     turnPointer = 0;
422
423     for(i = 0; i < MAX_PLAYERS; i++)
424         passedGoThisRound[i] = 0;
425
426     printf("\n=====\\n");
427     printf("ROUND %d\\n", round);
428     printf("=====\\n");
429
430     while(round <= MAX_ROUNDS)
431     {
432         playerIndex = turnOrder[turnPointer];
433
434         if(!game[0].players[playerIndex].bankrupt)
435         {
436             passedGo = playTurn(game, playerIndex);
437
438             if(passedGo)
439                 passedGoThisRound[playerIndex] = 1;
440         }
441
442         turnPointer = (turnPointer + 1) % MAX_PLAYERS;
443
444         /* has every player still playing passed GO this round? */
445         allPassed = 1;
446
447         for(i = 0; i < MAX_PLAYERS; i++)
448         {
449             if(game[0].players[i].bankrupt)
450                 continue;
451
452             if(!passedGoThisRound[i])
453             {
454                 allPassed = 0;
455                 break;
456             }
457         }
458
459         if(!allPassed)
460             continue;
461
462         processLoans(game);
463         processInsuranceExpiry(game);
464         ageProperties(game);
465         ageBuildings(game);
466         decrementEventTimers(game);
467         decrementModifiers(game);
468         enforceAntiSpeculation(game);
469
470         if(round % ECONOMY_CYCLE == 0)
471         {
472             triggerDisaster(game);
473             applyInflation(game);
474             reviewPropertyMarket(game, round);
475         }
476
477         if(round % EVENT_CYCLE == 0)
478         {
479             triggerEconomicEvent(game);
480             drawRegionalCard(game);
481         }
482
483         if(round % REGULATION_CYCLE == 0)
484         {
485             triggerGovernmentRegulation(game);
486         }
487
488         displayRoundSummary(game, round);
489         displayMarketConditions(game);
490     }
```

```

491         if(countSolventPlayers(game) <= 1)
492         {
493             printf("\nOnly one solvent player remains. Ending game.\n");
494             break;
495         }
496
497         round++;
498
499         if(round <= MAX_ROUNDS)
500         {
501             printf("\n=====\\n");
502             printf("ROUND %d\\n", round);
503             printf("=====\\n");
504         }
505
506         for(i = 0; i < MAX_PLAYERS; i++)
507             passedGoThisRound[i] = 0;
508     }
509 }
510
511 /* Whoever is still solvent wins. If several are still solvent at
512    round 500, the highest net worth wins (Rule 15). */
513 int determineWinner(GameState game[])
514 {
515     int i;
516     int winner;
517     int bestNetWorth;
518     int netWorth;
519
520     winner = -1;
521     bestNetWorth = 0;
522
523     for(i = 0; i < MAX_PLAYERS; i++)
524     {
525         if(game[i].players[i].bankrupt)
526             continue;
527
528         netWorth = calculateNetWorth(game, i);
529
530         if(winner == -1 || netWorth > bestNetWorth)
531         {
532             winner = i;
533             bestNetWorth = netWorth;
534         }
535     }
536
537     if(winner == -1)
538     {
539         /* edge case: everyone bankrupt at once - compare everyone anyway */
540         for(i = 0; i < MAX_PLAYERS; i++)
541         {
542             netWorth = calculateNetWorth(game, i);
543
544             if(winner == -1 || netWorth > bestNetWorth)
545             {
546                 winner = i;
547                 bestNetWorth = netWorth;
548             }
549         }
550     }
551
552     return winner;
553 }
554
555 void displayFinalResults(GameState game[])
556 {
557     int winner;
558     int i;
559     char cashBuf[32];
560     char propBuf[32];
561     char loanBuf[32];
562     char netBuf[32];
563
564     winner = determineWinner(game);
565
566     printf("\n=====\\n");
567     printf("GAME OVER\\n");
568     printf("=====\\n");
569
570     printf("Winner\\n%s\\n", game[winner].players[winner].name);
571
572     formatLKR(game[winner].players[winner].cash, cashBuf);

```

```
573     printf("Total Cash\nLKR %s\n", cashBuf);
574
575     formatLKR(calculatePropertyValue(game, winner) +
576             calculateBuildingValue(game, winner), propBuf);
577     printf("Total Property Value\nLKR %s\n", propBuf);
578
579     if(game[0].players[winner].loan.active)
580     {
581         formatLKR(game[0].players[winner].loan.amount, loanBuf);
582         printf("Outstanding Loans\nLKR %s\n", loanBuf);
583     }
584     else
585     {
586         printf("Outstanding Loans\nNone\n");
587     }
588
589     formatLKR(calculateNetWorth(game, winner), netBuf);
590     printf("Net Worth\nLKR %s\n", netBuf);
591     printf("=====\n");
592
593     printf("\nFinal Standings (all players)\n");
594     printf("=====\n");
595
596     for(i = 0; i < MAX_PLAYERS; i++)
597     {
598         printf("\n%s\n", game[0].players[i].name);
599         printf("Cash : LKR %d\n", game[0].players[i].cash);
600         printf("Net Worth : LKR %d\n", calculateNetWorth(game, i));
601         printf("Properties : %d\n", game[0].players[i].propertiesOwned);
602         printf("Railways : %d\n", game[0].players[i].railwaysOwned);
603         printf("Utilities : %d\n", game[0].players[i].utilitiesOwned);
604         if(game[0].players[i].bankrupt)
605         {
606             printf("Status : Bankrupt\n");
607         }
608         else
609         {
610             printf("Status : Active\n");
611         }
612     }
613 }
614
615 void startGame(GameState game[])
616 {
617     int turnOrder[MAX_PLAYERS];
618     char moneyBuffer[32];
619
620     srand((unsigned)time(NULL));
621
622     initializeBoard(game);
623     initializePlayers(game);
624     initEconomy(game);
625     initMarket(game);
626
627     printf("Player 1 : %s\n", game[0].players[0].name);
628     printf("Player 2 : %s\n", game[0].players[1].name);
629     printf("Player 3 : %s\n", game[0].players[2].name);
630     printf("Player 4 : %s\n", game[0].players[3].name);
631
632     formatLKR(START_MONEY, moneyBuffer);
633     printf("\nEach player begins with LKR %s.\n", moneyBuffer);
634
635     determineTurnOrder(game, turnOrder);
636
637     playGame(game, turnOrder);
638
639     displayFinalResults(game);
640 }
```

events.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "types.h"
4  #include "functions.h"
5
6  /* Keeps the loan interest rate inside its allowed range. */
7  void clampLoanInterest(GameState game[])
8  {
9      if(game[0].economy.loanInterestRate < LOAN_INTEREST_MIN)
10         game[0].economy.loanInterestRate = LOAN_INTEREST_MIN;
11
12         if(game[0].economy.loanInterestRate > LOAN_INTEREST_MAX)
13             game[0].economy.loanInterestRate = LOAN_INTEREST_MAX;
14     }
15
16     /* Moves the loan interest rate up or down by deltaPercent. */
17     void adjustLoanInterest(GameState game[], int deltaPercent)
18     {
19         game[0].economy.loanInterestRate += deltaPercent;
20         clampLoanInterest(game);
21     }
22
23     /* Appendix A: 20 National Event Cards, drawn when a player lands on
24        an EVENT square. currentCardIndex cycles 0-19 and wraps around,
25        which behaves the same as "draw the top card, put it at the
26        bottom" without needing a real deck data structure. */
27     void executeEvent(GameState game[], int playerIndex)
28     {
29         (void)playerIndex; /* not needed - these effects are global, not per-player */
30
31         printf("\n*** NATIONAL EVENT CARD ***\n");
32
33         switch(game[0].economy.currentCardIndex)
34         {
35             case 0:
36                 printf("Tourism Hype : Hotels earn double rent for 5 rounds.\n");
37                 addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 5);
38                 break;
39
40             case 1:
41                 printf("Fuel Shortage : Railway rent doubles for 5 rounds.\n");
42                 addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 5);
43                 break;
44
45             case 2:
46                 printf("Heavy Floods : a random coastal property is damaged.\n");
47                 triggerDisaster(game);
48                 break;
49
50             case 3:
51                 {
52                     int candidates[BOARD_SIZE];
53                     int count;
54                     int i;
55
56                     count = 0;
57
58                     for(i = 0; i < BOARD_SIZE; i++)
59                     {
60                         if(game[0].board[i].type == PROPERTY && game[0].board[i].property.owner != -1)
61                         {
62                             candidates[count] = i;
63                             count++;
64                         }
65                     }
66
67                     if(count > 0)
68                     {
69                         game[0].economy.closedPropertyIndex = candidates[rand() % count];
70                         game[0].economy.closedPropertyRoundsLeft = 2;
71
72                         printf("Political Rally : %s is closed for 2 rounds.\n",
73                             game[0].board[game[0].economy.closedPropertyIndex].name);
74                     }
75                     else
76                     {
77                         printf("Political Rally : no owned property to close.\n");
78                     }
79                     break;
80                 }
81         }
82     }

```

```

81
82     case 4:
83         printf("Stock Market Rise : All property values increase by 10%% for 15 rounds.\n");
84         addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
85         break;
86
87     case 5:
88         printf("Economic Downturn : Property values decrease by 15%% for 15 rounds.\n");
89         addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
90         break;
91
92     case 6:
93         printf("Housing Subsidy : House construction cost reduced by 30%% for 15 rounds.\n");
94         addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 15);
95         break;
96
97     case 7:
98         adjustLoanInterest(game, -LOAN_INTEREST_STEP);
99
100         printf("Interest Rate Cut : Loan interest reduced by %d%%. Now %d%%.\n",
101             LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
102         break;
103
104     case 8:
105         adjustLoanInterest(game, LOAN_INTEREST_STEP);
106
107         printf("Interest Rate Increase : Loan interest increased by %d%%. Now %d%%.\n",
108             LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
109         break;
110
111     case 9:
112     {
113         int i;
114
115         printf("Tax Amnesty : Each player receives LKR 2,000.\n");
116
117         for(i = 0; i < MAX_PLAYERS; i++)
118             receiveMoney(game, i, 2000);
119
120         break;
121     }
122
123     case 10:
124         printf("Power Failure : Utility income halved for 3 rounds.\n");
125         addModifier(game, MOD_UTIL_RENT, -1, -1, 50, 3);
126         break;
127
128     case 11:
129         printf("Foreign Funding : Commercial property values increase by 15%% for 15 rounds.\n");
130         addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 115, 15);
131         break;
132
133     case 12:
134         printf("Port Expansion : Railway station values increase by 20%% for 15 rounds.\n");
135         addModifier(game, MOD_RAIL_VALUE, -1, -1, 120, 15);
136         break;
137
138     case 13:
139         printf("Festival Season : Hotels receive 50%% additional rent for 5 rounds.\n");
140         addModifier(game, MOD_HOTEL_RENT, -1, -1, 150, 5);
141         break;
142
143     case 14:
144         printf("Labour Strike : Construction suspended for 2 rounds.\n");
145         game[0].economy.constructionSuspendedRoundsLeft = 2;
146         break;
147
148     case 15:
149         printf("Insurance Discount : Premiums reduced by 20%% for 10 rounds.\n");
150         addModifier(game, MOD_INSURANCE, -1, -1, 80, 10);
151         break;
152
153     case 16:
154     {
155         PropertyGroup group;
156         char groupBuffer[32];
157
158         group = (PropertyGroup)(rand() % NO_GROUP);
159         groupName(group, groupBuffer);
160
161         printf("Property Revaluation : %s properties appreciate by 15%% for 15 rounds.\n",
162             groupBuffer);

```

```

163         addModifier(game, MOD_GROUP_VALUE, group, -1, 115, 15);
164         break;
165     }
166
167     case 17:
168         printf("Currency Depreciation : Construction costs increase by 10%% for 15 rounds.\n");
169         addModifier(game, MOD_CONSTRUCTION, -1, -1, 110, 15);
170         break;
171
172     case 18:
173     {
174         int lucky;
175
176         /* Section 1.3: bankrupt players take no further part in the
177            game, so a grant must go to a solvent player. Pick a
178            random starting point and scan forward to the first
179            solvent player - at least one must exist, because a live
180            player drew this card. */
181         lucky = rand() % MAX_PLAYERS;
182
183         while(game[0].players[lucky].bankrupt)
184             lucky = (lucky + 1) % MAX_PLAYERS;
185
186         printf("Government Grant : %s receives LKR 5,000.\n",
187             game[0].players[lucky].name);
188
189         receiveMoney(game, lucky, 5000);
190         break;
191     }
192
193     case 19:
194         printf("National Disaster : a random developed property is damaged.\n");
195         triggerDisaster(game);
196         break;
197
198     default:
199         break;
200 }
201
202 game[0].economy.currentCardIndex = (game[0].economy.currentCardIndex + 1) % EVENT_CARD_COUNT;
203 }
204
205 /* Section 2.5: one of 8 events, every 15 rounds, affecting every player. */
206 void triggerEconomicEvent(GameState game[])
207 {
208     int choice;
209
210     choice = rand() % ECONOMIC_EVENT_COUNT;
211
212     printf("\n=== Economic Event ===\n");
213
214     switch(choice)
215     {
216     case 0:
217         printf("Tourism Boom\n");
218         printf("Hotels receive double rent for 15 rounds.\n");
219         printf("Southern Province properties increase in value by 15%%.\n");
220
221         addModifier(game, MOD_HOTEL_RENT, -1, -1, 200, 15);
222         addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 115, 15);
223         break;
224
225     case 1:
226         printf("Fuel Crisis\n");
227         printf("Railway rent doubles for 15 rounds.\n");
228         printf("Property development costs increase 20%% for 15 rounds.\n");
229
230         addModifier(game, MOD_RAIL_RENT, -1, -1, 200, 15);
231         addModifier(game, MOD_CONSTRUCTION, -1, -1, 120, 15);
232         break;
233
234     case 2:
235         printf("Heavy Monsoon\n");
236         printf("Insurance premiums increase for 15 rounds.\n");
237         printf("Coastal properties lose 10%% value.\n");
238
239         addModifier(game, MOD_INSURANCE, -1, -1, 115, 15);
240         addModifier(game, MOD_FLOOD_RISK, -1, -1, 100, 15);
241         addModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 90, 15);
242         break;
243
244     case 3:

```

```

245     printf("Economic Recession\n");
246     printf("Property values decrease 15%. Rent decreases 10%%.\n");
247     printf("Loan interest increases by 15%%.\n");
248
249     addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 85, 15);
250     addModifier(game, MOD_RENT_GLOBAL, -1, -1, 90, 15);
251     addModifier(game, MOD_RECESSION, -1, -1, 100, 15);
252
253     game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, 15);
254     clampLoanInterest(game);
255     break;
256
257 case 4:
258     printf("Stock Market Boom\n");
259     printf("Property values increase 10%. Loan interest decreases 10%%.\n");
260
261     addModifier(game, MOD_VALUE_GLOBAL, -1, -1, 110, 15);
262
263     game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, -10);
264     clampLoanInterest(game);
265     break;
266
267 case 5:
268     printf("Government Housing Programme\n");
269     printf("House construction costs reduce 25%%.\n");
270
271     addModifier(game, MOD_CONSTRUCTION, -1, -1, 75, 15);
272     break;
273
274 case 6:
275     printf("Foreign Investment\n");
276     printf("Commercial properties increase 20%%.\n");
277
278     addModifier(game, MOD_GROUP_VALUE, ORANGE, -1, 120, 15);
279     break;
280
281 case 7:
282     printf("Political Unrest\n");
283     printf("Hotel rent drops by 50%% for 15 rounds.\n");
284
285     addModifier(game, MOD_HOTEL_RENT, -1, -1, 50, 15);
286     addModifier(game, MOD_RIOT_RISK, -1, -1, 100, 15);
287     addModifier(game, MOD_BI_CLAIMS, -1, -1, 100, 15);
288     break;
289
290 default:
291     break;
292 }
293 }
294
295 /* Section 2.7: one of 8 regulations, every 20 rounds. */
296 void triggerGovernmentRegulation(GameState game[])
297 {
298     int choice;
299
300     choice = rand() % REGULATION_COUNT;
301
302     printf("\n== Government Regulation ==\n");
303
304     switch(choice)
305     {
306     case 0:
307         game[0].economy.incomeTaxRate = applyRate(game[0].economy.incomeTaxRate, 50);
308
309         if(game[0].economy.incomeTaxRate > TAX_RATE_MAX)
310             game[0].economy.incomeTaxRate = TAX_RATE_MAX;
311
312         printf("Increase Property Tax\n");
313         printf("Income Tax rate increased by 50%. Now %d%%.\n", game[0].economy.incomeTaxRate);
314         break;
315
316     case 1:
317         adjustLoanInterest(game, -LOAN_INTEREST_STEP);
318
319         printf("Reduce Loan Interest\n");
320         printf("Interest decreased by %d%. Now %d%%.\n",
321             LOAN_INTEREST_STEP, game[0].economy.loanInterestRate);
322         break;
323
324     case 2:
325         printf("Housing Subsidy\n");
326         printf("House construction costs reduced by 30%% for 20 rounds.\n");

```

```
327
328     addModifier(game, MOD_CONSTRUCTION, -1, -1, 70, 20);
329     break;
330
331     case 3:
332     {
333         int i;
334         int tax;
335
336         printf("Luxury Property Tax\n");
337         printf("Hotels pay a maintenance tax of 25% of their value.\n");
338
339         for(i = 0; i < BOARD_SIZE; i++)
340         {
341             if(game[0].board[i].type == PROPERTY && game[0].board[i].property.hotel &&
342                game[0].board[i].property.owner != -1)
343             {
344                 tax = (currentMarketValue(game, i) * HOTEL_LUXURY_TAX_PERCENT) / 100;
345
346                 printf("%s pays LKR %d luxury tax on %s.\n",
347                     game[0].players[game[0].board[i].property.owner].name,
348                     tax, game[0].board[i].name);
349
350                 payMoney(game, game[0].board[i].property.owner, tax);
351             }
352         }
353         break;
354     }
355
356     case 4:
357         printf("Railway Modernization\n");
358         printf("Railway rents increase 25% for 20 rounds.\n");
359
360         addModifier(game, MOD_RAIL_RENT, -1, -1, 125, 20);
361         break;
362
363     case 5:
364         printf("Electricity Tariff Revision\n");
365         printf("Utility rents increase 20% for 20 rounds.\n");
366
367         addModifier(game, MOD_UTIL_RENT, -1, -1, 120, 20);
368         break;
369
370     case 6:
371         printf("Insurance Regulation\n");
372         printf("Insurance premiums decrease 15% for 20 rounds. Coverage unchanged.\n");
373
374         addModifier(game, MOD_INSURANCE, -1, -1, 85, 20);
375         break;
376
377     case 7:
378         printf("Anti-Speculation Act\n");
379         printf("Players may now own at most three undeveloped properties.\n");
380
381         game[0].economy.antiSpeculationActive = 1;
382         break;
383
384     default:
385         break;
386 }
387 }
388
389 /* Decrements the non-modifier event timers - the modifier list itself
390    is ticked down separately by decrementModifiers(). */
391 void decrementEventTimers(GameState game[])
392 {
393     if(game[0].economy.constructionSuspendedRoundsLeft > 0)
394         game[0].economy.constructionSuspendedRoundsLeft--;
395
396     if(game[0].economy.closedPropertyRoundsLeft > 0)
397     {
398         game[0].economy.closedPropertyRoundsLeft--;
399         if(game[0].economy.closedPropertyRoundsLeft == 0)
400             game[0].economy.closedPropertyIndex = -1;
401     }
402 }
```

economy.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "types.h"
5  #include "functions.h"
6
7  void initEconomy(GameState game[])
8  {
9      game[0].economy.inflationRate = 0;
10     game[0].economy.loanInterestRate = LOAN_INTEREST_RATE;
11
12     game[0].economy.constructionSuspendedRoundsLeft = 0;
13
14     game[0].economy.closedPropertyIndex = -1;
15     game[0].economy.closedPropertyRoundsLeft = 0;
16
17     game[0].economy.incomeTaxRate = INCOME_TAX_RATE;
18
19     game[0].economy.antiSpeculationActive = 0;
20
21     game[0].economy.currentCardIndex = 0;
22
23     game[0].economy.modifierCount = 0;
24 }
25
26 /* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */
27 int applyRate(int oldValue, int ratePercent)
28 {
29     int newValue;
30
31     newValue = oldValue + (oldValue * ratePercent) / 100;
32
33     if(newValue < 1)
34         newValue = 1;
35
36     return newValue;
37 }
38
39 /* Adds a timed modifier (Rule-LK 30-35). group/index are -1 when not
40    relevant. Effects accumulate: two active +25% rents give
41    roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */
42 void addSourcedModifier(GameState game[], ModifierType type, int group,
43                        int index, int percent, int roundsLeft,
44                        ModifierSource source)
45 {
46     if(game[0].economy.modifierCount >= MAX_MODIFIERS)
47         return;
48
49     game[0].economy.modifiers[game[0].economy.modifierCount].type = type;
50     game[0].economy.modifiers[game[0].economy.modifierCount].group = group;
51     game[0].economy.modifiers[game[0].economy.modifierCount].index = index;
52     game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent;
53     game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft;
54     game[0].economy.modifiers[game[0].economy.modifierCount].source = source;
55
56     game[0].economy.modifierCount++;
57 }
58
59 /* Events, national cards and regulations use this - they are all
60    generic effects, not Market Boom/Decline or Regional cards. */
61 void addModifier(GameState game[], ModifierType type, int group, int index,
62                int percent, int roundsLeft)
63 {
64     addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL);
65 }
66
67 /* Effective percentage multiplier from every active matching modifier,
68    multiplied together (Rule-LK 34). Returns 100 when none match.
69    A modifier with group == -1 is global and applies to every group,
70    so it matches regardless of the requested group (and likewise for
71    index == -1). */
72 int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
73 {
74     int mult;
75     int i;
76
77     mult = 100;
78
79     for(i = 0; i < game[0].economy.modifierCount; i++)
80     {

```

```

81         if(game[0].economy.modifiers[i].type != type)
82             continue;
83         if(group != -1 && game[0].economy.modifiers[i].group != -1 &&
84            game[0].economy.modifiers[i].group != group)
85             continue;
86         if(index != -1 && game[0].economy.modifiers[i].index != -1 &&
87            game[0].economy.modifiers[i].index != index)
88             continue;
89
90         mult = (mult * game[0].economy.modifiers[i].percent) / 100;
91     }
92
93     return mult;
94 }
95
96 /* Whether at least one matching modifier is currently active - used
97    for flag-style modifiers such as flood risk or recession. */
98 int isModifierActive(GameState game[], ModifierType type, int group, int index)
99 {
100     int i;
101
102     for(i = 0; i < game[0].economy.modifierCount; i++)
103     {
104         if(game[0].economy.modifiers[i].type != type)
105             continue;
106         if(group != -1 && game[0].economy.modifiers[i].group != group)
107             continue;
108         if(index != -1 && game[0].economy.modifiers[i].index != index)
109             continue;
110
111         return 1;
112     }
113
114     return 0;
115 }
116
117 /* The effective tax rate used by the Income Tax and Community
118    Development Fund squares. Starts at the given base rate (15% /
119    10%) and is scaled by the currently active global value
120    modifiers, so market fluctuations move the rate up or down. */
121 int currentTaxRatePercent(GameState game[], int baseRate)
122 {
123     return (baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100;
124 }
125
126 /* Ticks every active modifier down and removes the expired ones. */
127 void decrementModifiers(GameState game[])
128 {
129     int i;
130
131     i = 0;
132
133     while(i < game[0].economy.modifierCount)
134     {
135         game[0].economy.modifiers[i].roundsLeft--;
136
137         if(game[0].economy.modifiers[i].roundsLeft <= 0)
138         {
139             int j;
140
141             for(j = i; j < game[0].economy.modifierCount - 1; j++)
142                 game[0].economy.modifiers[j] = game[0].economy.modifiers[j + 1];
143
144             game[0].economy.modifierCount--;
145         }
146         else
147         {
148             i++;
149         }
150     }
151 }
152
153 /* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */
154 void formatLKR(int amount, char out[])
155 {
156     char digits[32];
157     int len;
158     int i;
159     int w;
160     int neg;
161
162     neg = 0;

```

```

163
164     if(amount < 0)
165     {
166         neg = 1;
167         amount = -amount;
168     }
169
170     sprintf(digits, "%d", amount);
171     len = strlen(digits);
172
173     w = 0;
174
175     if(neg)
176         out[w++] = '-';
177
178     for(i = 0; i < len; i++)
179     {
180         if(i > 0 && (len - i) % 3 == 0)
181             out[w++] = ',';
182
183         out[w++] = digits[i];
184     }
185
186     out[w] = '\0';
187 }
188
189 /* Every 10 rounds, a random inflation rate is applied to every
190    property's price, rent, and construction costs. */
191 void applyInflation(GameState game[])
192 {
193     int possibleRates[6] = {-3, 0, 2, 5, 8, 12};
194     int rate;
195     int i;
196
197     rate = possibleRates[rand() % 6];
198     game[0].economy.inflationRate = rate;
199
200     printf("\n=== Inflation Update ===\n");
201     printf("New Inflation Rate : %d%\n", rate);
202
203     for(i = 0; i < BOARD_SIZE; i++)
204     {
205         if(game[0].board[i].type != PROPERTY &&
206            game[0].board[i].type != RAILWAY &&
207            game[0].board[i].type != UTILITY)
208         {
209             continue;
210         }
211
212         game[0].board[i].property.purchasePrice =
213             applyRate(game[0].board[i].property.purchasePrice, rate);
214
215         game[0].board[i].property.mortgageValue =
216             applyRate(game[0].board[i].property.mortgageValue, rate);
217
218         if(game[0].board[i].type == PROPERTY)
219         {
220             game[0].board[i].property.baseRent =
221                 applyRate(game[0].board[i].property.baseRent, rate);
222
223             game[0].board[i].property.houseCost =
224                 applyRate(game[0].board[i].property.houseCost, rate);
225
226             game[0].board[i].property.hotelCost =
227                 applyRate(game[0].board[i].property.hotelCost, rate);
228         }
229     }
230
231     game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, rate);
232
233     printf("New Loan Interest Rate : %d%\n", game[0].economy.loanInterestRate);
234 }
235
236 /* A property's real current worth: purchase price, minus age
237    depreciation, adjusted by any active market conditions. Reused
238    everywhere a "real price" is needed - buying, insurance, repairs,
239    renovation, net worth - so all of them stay consistent. */
240 /* Rule-LK 11-13, 24: the current market value of a property, after
241    age depreciation and every value modifier currently active
242    (Rule-LK 34 - all matching modifiers multiply together). */
243 int currentMarketValue(GameState game[], int propIndex)
244 {

```

```

245     int price;
246     int depreciation;
247     int mult;
248
249     price = game[0].board[propIndex].property.purchasePrice;
250     depreciation = game[0].board[propIndex].property.depreciation;
251
252     price = price - (price * depreciation) / 100;
253
254     mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);
255
256     if(game[0].board[propIndex].type == PROPERTY)
257         mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,
258                                           game[0].board[propIndex].property.group, -1)) / 100;
259
260     if(game[0].board[propIndex].type == RAILWAY)
261         mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100;
262
263     mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100;
264
265     price = (price * mult) / 100;
266
267     return price;
268 }
269
270 /* Rule-LK 15, 16: properties get older every round, and lose value
271    past age 50. Prints the required "Property Depreciation" message
272    (Section 5) whenever a property's depreciation actually changes. */
273 void ageProperties(GameState game[])
274 {
275     int i;
276     int roundsOver50;
277     int newDepreciation;
278     int oldDepreciation;
279
280     for(i = 0; i < BOARD_SIZE; i++)
281     {
282         if(game[0].board[i].type != PROPERTY)
283             continue;
284
285         game[0].board[i].property.age++;
286
287         if(game[0].board[i].property.age > PROPERTY_AGE_LIMIT)
288         {
289             oldDepreciation = game[0].board[i].property.depreciation;
290
291             roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;
292             newDepreciation = roundsOver50 / DEPRECIATION_STEP;
293
294             if(newDepreciation > MAX_DEPRECIATION)
295                 newDepreciation = MAX_DEPRECIATION;
296
297             game[0].board[i].property.depreciation = newDepreciation;
298
299             if(newDepreciation != oldDepreciation)
300             {
301                 printf("\nProperty Depreciation\n");
302                 printf("Property\n%s\n", game[0].board[i].name);
303                 printf("has depreciated by %d%%.\n", newDepreciation);
304                 printf("Current Value\nLKR %d\n", currentMarketValue(game, i));
305             }
306         }
307     }
308 }
309
310 /* Rule-LK 17: landing on your own aged property offers a renovation. */
311 void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
312 {
313     int cost;
314
315     if(game[0].board[propIndex].property.depreciation <= 0)
316         return;
317
318     if(!shouldRenovateAgeDepreciation(game, playerIndex,
319                                       game[0].board[propIndex].property.depreciation))
320     {
321         return;
322     }
323
324     cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;
325
326     if(game[0].players[playerIndex].cash < cost)

```

```

327         return;
328
329     payMoney(game, playerIndex, cost);
330
331     game[0].board[propIndex].property.age = 0;
332     game[0].board[propIndex].property.depreciation = 0;
333
334     game[0].board[propIndex].property.baseRent =
335         applyRate(game[0].board[propIndex].property.baseRent, RENOVATION_RENT_BOOST);
336
337     printf("\n%s renovated %s.\n",
338         game[0].players[playerIndex].name, game[0].board[propIndex].name);
339
340     printf("Renovation Cost : LKR %d\n", cost);
341 }
342
343 /* Table 3: rent collected as a percentage of building condition. */
344 int rentConditionPercent(int condition)
345 {
346     if(condition >= 90)
347         return 100;
348     if(condition >= 75)
349         return 90;
350     if(condition >= 50)
351         return 75;
352     if(condition >= 25)
353         return 50;
354
355     return 0; /* below 25% - building closed */
356 }
357
358 /* Rule-LK 25, 28: building condition wears down every round, and
359    20+ rounds of neglect causes lasting structural damage. */
360 void ageBuildings(GameState game[])
361 {
362     int i;
363
364     for(i = 0; i < BOARD_SIZE; i++)
365     {
366         if(game[0].board[i].type != PROPERTY)
367             continue;
368
369         /* Business Interruption's 5-round lost-income countdown runs
370            even if the property isn't currently developed with houses
371            (it still has a hotel that just isn't earning anything). */
372         if(game[0].board[i].property.lostIncomeRoundsLeft > 0)
373             game[0].board[i].property.lostIncomeRoundsLeft--;
374
375         if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
376             continue;
377
378         game[0].board[i].property.condition -= 2;
379
380         if(game[0].board[i].property.condition < 0)
381             game[0].board[i].property.condition = 0;
382
383         game[0].board[i].property.roundsSinceMaintenance++;
384
385         if(game[0].board[i].property.roundsSinceMaintenance > NEGLECT_ROUNDS &&
386            !game[0].board[i].property.structurallyDamaged)
387         {
388             game[0].board[i].property.structurallyDamaged = 1;
389
390             game[0].board[i].property.preDamagePurchasePrice =
391                 game[0].board[i].property.purchasePrice;
392             game[0].board[i].property.preDamageBaseRent =
393                 game[0].board[i].property.baseRent;
394
395             game[0].board[i].property.purchasePrice =
396                 game[0].board[i].property.purchasePrice -
397                 (game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;
398
399             game[0].board[i].property.baseRent =
400                 game[0].board[i].property.baseRent -
401                 (game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;
402
403             game[0].board[i].property.maintenanceCostMultiplierPercent = DAMAGE_REPAIR_MULTIPLIER;
404             game[0].board[i].property.condition = 0;
405
406             printf("\n%s has suffered structural damage from neglect!\n",
407                 game[0].board[i].name);
408         }

```

```

409     }
410 }
411
412 /* Rule-LK 27: restores a building's condition to 100% for a fee. */
413 void performMaintenance(GameState game[], int playerIndex)
414 {
415     int i;
416     int baseCost;
417     int cost;
418
419     for(i = 0; i < BOARD_SIZE; i++)
420     {
421         if(game[0].board[i].type != PROPERTY)
422             continue;
423
424         if(game[0].board[i].property.owner != playerIndex)
425             continue;
426
427         if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
428             continue;
429
430         if(game[0].board[i].property.structurallyDamaged)
431             continue; /* needs a full renovation instead */
432
433         if(game[0].board[i].property.condition >= 100)
434             continue;
435
436         if(game[0].board[i].property.hotel)
437             baseCost = (game[0].board[i].property.hotelCost * 8) / 100;
438         else
439             baseCost = (game[0].board[i].property.houseCost * 5) / 100;
440
441         cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierPercent) / 100;
442
443         if(!shouldMaintain(game, playerIndex, game[0].board[i].property.condition, cost))
444             continue;
445
446         if(game[0].players[playerIndex].cash < cost)
447             continue;
448
449         payMoney(game, playerIndex, cost);
450
451         game[0].board[i].property.condition = 100;
452         game[0].board[i].property.roundsSinceMaintenance = 0;
453
454         printf("\n%s performed maintenance on %s.\n",
455             game[0].players[playerIndex].name, game[0].board[i].name);
456
457         printf("Maintenance Cost : LKR %d\n", cost);
458     }
459 }
460
461 /* Rule-LK 29: fully restores a structurally damaged building. */
462 void renovateStructuralDamage(GameState game[], int playerIndex)
463 {
464     int i;
465     int replacementValue;
466     int cost;
467
468     for(i = 0; i < BOARD_SIZE; i++)
469     {
470         if(game[0].board[i].type != PROPERTY)
471             continue;
472
473         if(game[0].board[i].property.owner != playerIndex)
474             continue;
475
476         if(!game[0].board[i].property.structurallyDamaged)
477             continue;
478
479         if(game[0].board[i].property.hotel)
480             replacementValue = game[0].board[i].property.hotelCost;
481         else
482             replacementValue = game[0].board[i].property.houseCost;
483
484         cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;
485
486         if(game[0].players[playerIndex].cash < cost)
487             continue;
488
489         payMoney(game, playerIndex, cost);
490

```

```
491     game[0].board[i].property.purchasePrice = game[0].board[i].property.preDamagePurchasePrice;
492     game[0].board[i].property.baseRent = game[0].board[i].property.preDamageBaseRent;
493     game[0].board[i].property.condition = 100;
494     game[0].board[i].property.structurallyDamaged = 0;
495     game[0].board[i].property.maintenanceCostMultiplierPercent = 100;
496     game[0].board[i].property.roundsSinceMaintenance = 0;
497
498     printf("\n%s fully renovated %s after structural damage.\n",
499           game[0].players[playerIndex].name, game[0].board[i].name);
500
501     printf("Renovation Cost : LKR %d\n", cost);
502 }
503 }
```

market.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "types.h"
5  #include "functions.h"
6
7  void initMarket(GameState game[])
8  {
9      int i;
10
11      for(i = 0; i < NO_GROUP; i++)
12          game[0].economy.groupCooldownUntilRound[i] = 0;
13
14      game[0].economy.lastBoomGroup = -1;
15      game[0].economy.lastDeclineGroup = -1;
16  }
17
18  void groupName(PropertyGroup group, char buffer[])
19  {
20      if(group == BROWN)
21          strcpy(buffer, "Brown");
22      else if(group == LIGHT_BLUE)
23          strcpy(buffer, "Light Blue");
24      else if(group == PINK)
25          strcpy(buffer, "Pink");
26      else if(group == ORANGE)
27          strcpy(buffer, "Orange");
28      else if(group == RED)
29          strcpy(buffer, "Red");
30      else if(group == YELLOW)
31          strcpy(buffer, "Yellow");
32      else if(group == GREEN)
33          strcpy(buffer, "Green");
34      else if(group == DARK_BLUE)
35          strcpy(buffer, "Dark Blue");
36      else
37          strcpy(buffer, "Unknown");
38  }
39
40  /* Picks a random group that isn't on cooldown and isn't one of the
41     two groups to avoid (Rule-LK 30, 33). */
42  PropertyGroup pickEligibleGroup(GameState game[], int currentRound, int avoid1, int avoid2)
43  {
44      int candidates[NO_GROUP];
45      int count;
46      int g;
47
48      count = 0;
49
50      for(g = 0; g < NO_GROUP; g++)
51      {
52          if(g == avoid1 || g == avoid2)
53              continue;
54
55          if(game[0].economy.groupCooldownUntilRound[g] > currentRound)
56              continue;
57
58          candidates[count] = g;
59          count++;
60      }
61
62      if(count == 0)
63          return (PropertyGroup)((avoid1 + 1) % NO_GROUP); /* rare fallback */
64
65      return (PropertyGroup)candidates[rand() % count];
66  }
67
68  /* Rule-LK 30-34: every 10 rounds, one group booms and a different
69     one declines, each for 10 rounds. All effects are temporary
70     modifiers (Rule-LK 34) so no base value or cost is ever changed. */
71  void reviewPropertyMarket(GameState game[], int currentRound)
72  {
73      PropertyGroup boomGroup;
74      PropertyGroup declineGroup;
75      char boomName[12];
76      char declineName[12];
77
78      boomGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastBoomGroup, -1);
79      declineGroup = pickEligibleGroup(game, currentRound, game[0].economy.lastDeclineGroup, boomGroup);
80  }

```

```

81     groupName(boomGroup, boomName);
82     groupName(declineGroup, declineName);
83
84     printf("\n=== Property Market Review ===\n");
85     printf("Market Boom : %s (values +20%, rent +25%) for 10 rounds.\n", boomName);
86     printf("Market Decline : %s (values -15%, rent -20%) for 10 rounds.\n", declineName);
87
88     addSourcedModifier(game, MOD_GROUP_VALUE, boomGroup, -1, 120, 10, SRC_BOOM);
89     addSourcedModifier(game, MOD_GROUP_RENT, boomGroup, -1, 125, 10, SRC_BOOM);
90
91     addSourcedModifier(game, MOD_GROUP_VALUE, declineGroup, -1, 85, 10, SRC_DECLINE);
92     addSourcedModifier(game, MOD_GROUP_RENT, declineGroup, -1, 80, 10, SRC_DECLINE);
93
94     /* Rule-LK 31/32/34: the boom group's mortgage values rise by 15%,
95        direct purchase prices rise by 15% and construction costs rise
96        by 10%; the decline group's mortgage values fall by 10% and
97        auction starting prices fall by 25%. All are scoped to the
98        affected group. */
99     addSourcedModifier(game, MOD_MARKET_MORTGAGE, boomGroup, -1, 115, 10, SRC_BOOM);
100    addSourcedModifier(game, MOD_PURCHASE_PRICE, boomGroup, -1, 115, 10, SRC_BOOM);
101    addSourcedModifier(game, MOD_CONSTRUCTION, boomGroup, -1, 110, 10, SRC_BOOM);
102
103    addSourcedModifier(game, MOD_MARKET_MORTGAGE, declineGroup, -1, 90, 10, SRC_DECLINE);
104    addSourcedModifier(game, MOD_AUCTION_PRICE, declineGroup, -1, 75, 10, SRC_DECLINE);
105
106    game[0].economy.groupCooldownUntilRound[boomGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;
107    game[0].economy.groupCooldownUntilRound[declineGroup] = currentRound + MARKET_COOLDOWN_ROUNDS;
108
109    game[0].economy.lastBoomGroup = boomGroup;
110    game[0].economy.lastDeclineGroup = declineGroup;
111 }
112
113 /* Table 4: one card every 15 rounds, active for 15 rounds. */
114 void drawRegionalCard(GameState game[])
115 {
116     int choice;
117
118     choice = rand() % REGIONAL_CARD_COUNT;
119
120     printf("\n=== Regional Development Card ===\n");
121
122     switch(choice)
123     {
124         case 0:
125             printf("Southern Tourism Boom : Galle Fort, Unawatuna and "
126                  "Hikkaduwa rental income +40%.\n");
127             addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 140, 15, SRC_REGIONAL);
128             break;
129
130         case 1:
131             printf("Port City Expansion : Pettah and Maradana values +25%.\n");
132             addSourcedModifier(game, MOD_GROUP_VALUE, BROWN, -1, 125, 15, SRC_REGIONAL);
133             addSourcedModifier(game, MOD_INDEX_VALUE, -1, 5, 125, 15, SRC_REGIONAL); /* Colombo Fort Station */
134             break;
135
136         case 2:
137             printf("IT Industry Growth : Maharagama, Nugegoda and "
138                  "Kottawa values +20%.\n");
139             addSourcedModifier(game, MOD_GROUP_VALUE, PINK, -1, 120, 15, SRC_REGIONAL);
140             break;
141
142         case 3:
143             printf("Northern Development Programme : Jaffna Town, Nallur "
144                  "and Trincomalee values +30%.\n");
145             addSourcedModifier(game, MOD_GROUP_VALUE, GREEN, -1, 130, 15, SRC_REGIONAL);
146             break;
147
148         case 4:
149             printf("Tea Export Boom : Nuwara Eliya value +35%.\n");
150             addSourcedModifier(game, MOD_INDEX_VALUE, -1, 37, 135, 15, SRC_REGIONAL);
151             break;
152
153         case 5:
154             printf("Airport Expansion : Negombo, Katunayake and "
155                  "Ja-Ela rents +30%.\n");
156             addSourcedModifier(game, MOD_GROUP_RENT, ORANGE, -1, 130, 15, SRC_REGIONAL);
157             break;
158
159         case 6:
160             printf("University City Growth : Peradeniya and "
161                  "Kandy City values +20%.\n");
162             addSourcedModifier(game, MOD_GROUP_VALUE, RED, -1, 120, 15, SRC_REGIONAL);

```

```

163         break;
164
165     case 7:
166         printf("Beach Pollution : Southern coastal rents -30%%.\n");
167         addSourcedModifier(game, MOD_GROUP_RENT, YELLOW, -1, 70, 15, SRC_REGIONAL);
168         break;
169
170     case 8:
171         printf("Flood Damage : Low-lying coastal properties lose 20%% value.\n");
172         addSourcedModifier(game, MOD_GROUP_VALUE, YELLOW, -1, 80, 15, SRC_REGIONAL);
173         break;
174
175     case 9:
176         printf("Transport Strike : Railway revenue reduced by 40%%.\n");
177         addSourcedModifier(game, MOD_RAIL_RENT, -1, -1, 60, 15, SRC_REGIONAL);
178         break;
179
180     case 10:
181         printf("Electricity Tariff Increase : Utility rent +25%%.\n");
182         addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 125, 15, SRC_REGIONAL);
183         break;
184
185     case 11:
186         printf("Water Shortage : Utility revenue +20%%.\n");
187         addSourcedModifier(game, MOD_UTIL_RENT, -1, -1, 120, 15, SRC_REGIONAL);
188         addSourcedModifier(game, MOD_INDEX_VALUE, -1, 13, 90, 15, SRC_REGIONAL);
189         addSourcedModifier(game, MOD_INDEX_VALUE, -1, 29, 90, 15, SRC_REGIONAL);
190         break;
191
192     default:
193         break;
194 }
195 }
196
197 /* Prints a section title followed by a separator line. */
198 void printSectionHeader(char title[])
199 {
200     printf("%s\n", title);
201     printf("-----\n");
202 }
203
204 /* Prints a Market Boom or Market Decline block. */
205 void printBoomDecline(char title[], char groupNameBuffer[], int percent, int roundsLeft)
206 {
207     printSectionHeader(title);
208     printf("%s (%+d%%)\n", groupNameBuffer, percent - 100);
209     printf("Rounds Remaining : %d\n", roundsLeft);
210 }
211
212 /* Writes one Regional Development card line into out. Returns 1 if
213 the modifier has a line to show, 0 otherwise. */
214 int describeRegionalCondition(GameState game[], ActiveModifier mod, char out[])
215 {
216     char name[48];
217
218     if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
219     {
220         if(mod.group >= 0 && mod.group < NO_GROUP)
221             groupName((PropertyGroup)mod.group, name);
222         else
223             strcpy(name, "Unknown");
224
225         if(mod.type == MOD_GROUP_VALUE)
226             sprintf(out, "%s (%+d%%)", name, mod.percent - 100);
227         else
228             sprintf(out, "%s rents (%+d%%)", name, mod.percent - 100);
229     }
230     else if(mod.type == MOD_INDEX_VALUE)
231     {
232         sprintf(out, "%s (%+d%%)",
233             game[0].board[mod.index].name, mod.percent - 100);
234     }
235     else if(mod.type == MOD_RAIL_RENT)
236     {
237         sprintf(out, "Railway rents (%+d%%)", mod.percent - 100);
238     }
239     else if(mod.type == MOD_UTIL_RENT)
240     {
241         sprintf(out, "Utility rents (%+d%%)", mod.percent - 100);
242     }
243     else
244     {

```

```
245     return 0;
246 }
247
248     return 1;
249 }
250
251 /* Prints the "Regional Development" section. Returns 1 if at least
252    one card effect was shown. */
253 int printRegionalSection(GameState game[])
254 {
255     int i;
256     int shown;
257
258     shown = 0;
259
260     for(i = 0; i < game[0].economy.modifierCount; i++)
261     {
262         ActiveModifier mod;
263         char line[100];
264
265         mod = game[0].economy.modifiers[i];
266
267         if(mod.source != SRC_REGIONAL)
268             continue;
269
270         if(!describeRegionalCondition(game, mod, line))
271             continue;
272
273         if(!shown)
274         {
275             printSectionHeader("Regional Development");
276             shown = 1;
277         }
278
279         printf("%s\n", line);
280         printf("Rounds Remaining : %d\n", mod.roundsLeft);
281     }
282
283     return shown;
284 }
285
286 /* Writes one "Other Active Conditions" line into out. Returns 1 if
287    the modifier has a line to show, 0 otherwise. */
288 int describeOtherCondition(GameState game[], ActiveModifier mod, char out[])
289 {
290     char name[32];
291
292     if(mod.type == MOD_GROUP_VALUE || mod.type == MOD_GROUP_RENT)
293     {
294         groupName((PropertyGroup)mod.group, name);
295
296         if(mod.type == MOD_GROUP_VALUE)
297             sprintf(out, "%s : values x%d%% (%d rounds remaining)",
298                 name, mod.percent, mod.roundsLeft);
299         else
300             sprintf(out, "%s : rents x%d%% (%d rounds remaining)",
301                 name, mod.percent, mod.roundsLeft);
302     }
303     else if(mod.type == MOD_INDEX_VALUE)
304     {
305         sprintf(out, "%s : value x%d%% (%d rounds remaining)",
306             game[0].board[mod.index].name, mod.percent, mod.roundsLeft);
307     }
308     else if(mod.type == MOD_RAIL_VALUE)
309     {
310         sprintf(out, "Railway values : x%d%% (%d rounds remaining)",
311             mod.percent, mod.roundsLeft);
312     }
313     else if(mod.type == MOD_RAIL_RENT)
314     {
315         sprintf(out, "Railway rents : x%d%% (%d rounds remaining)",
316             mod.percent, mod.roundsLeft);
317     }
318     else if(mod.type == MOD_UTIL_RENT)
319     {
320         sprintf(out, "Utility rents : x%d%% (%d rounds remaining)",
321             mod.percent, mod.roundsLeft);
322     }
323     else
324     {
325         return 0;
326     }
327 }
```

```

327
328     return 1;
329 }
330
331 /* Prints the "Other Active Conditions" section (events and
332    government regulations still running). */
333 void printOtherConditions(GameState game[])
334 {
335     int i;
336     int shown;
337
338     shown = 0;
339
340     for(i = 0; i < game[0].economy.modifierCount; i++)
341     {
342         ActiveModifier mod;
343         char line[100];
344
345         mod = game[0].economy.modifiers[i];
346
347         if(mod.source != SRC_GENERAL)
348             continue;
349
350         if(!describeOtherCondition(game, mod, line))
351             continue;
352
353         if(!shown)
354         {
355             printSectionHeader("Other Active Conditions");
356             shown = 1;
357         }
358
359         printf("%s\n", line);
360     }
361 }
362
363 /* Rule-LK 36: shows the currently active market conditions at the
364    end of every round, in the Section 5 format. */
365 void displayMarketConditions(GameState game[])
366 {
367     int i;
368     int shown;
369
370     printf("\n===== \n");
371     printf("Current Market Conditions\n");
372     printf("===== \n");
373
374     shown = 0;
375
376     /* Market Boom (Rule-LK 31) */
377     for(i = 0; i < game[0].economy.modifierCount; i++)
378     {
379         char name[32];
380
381         if(game[0].economy.modifiers[i].source != SRC_BOOM ||
382            game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
383             continue;
384
385         groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
386         printBoomDecline("Market Boom", name,
387                          game[0].economy.modifiers[i].percent,
388                          game[0].economy.modifiers[i].roundsLeft);
389
390         shown = 1;
391         break;
392     }
393
394     /* Market Decline (Rule-LK 32) */
395     for(i = 0; i < game[0].economy.modifierCount; i++)
396     {
397         char name[32];
398
399         if(game[0].economy.modifiers[i].source != SRC_DECLINE ||
400            game[0].economy.modifiers[i].type != MOD_GROUP_VALUE)
401             continue;
402
403         groupName((PropertyGroup)game[0].economy.modifiers[i].group, name);
404         printBoomDecline("Market Decline", name,
405                          game[0].economy.modifiers[i].percent,
406                          game[0].economy.modifiers[i].roundsLeft);
407
408         shown = 1;
409         break;
410     }

```

```
409
410     /* Regional Development Card effects (Table 4) */
411     if(printRegionalSection(game))
412         shown = 1;
413
414     if(!shown)
415         printf("No active market booms, declines or regional conditions right now.\n");
416
417     /* Other active event/regulation effects */
418     printOtherConditions(game);
419
420     printSectionHeader("Inflation");
421     printf("%+d%%\n", game[0].economy.inflationRate);
422
423     printSectionHeader("Current Loan Interest");
424     printf("%d%%\n", game[0].economy.loanInterestRate);
425
426     printSectionHeader("Current Income Tax Rate");
427     printf("%d%%\n", game[0].economy.incomeTaxRate);
428
429     printf("=====\n");
430 }
```

bank.c

```

1  #include <stdio.h>
2  #include "types.h"
3  #include "functions.h"
4
5  /* Rule-LK: each colour group has a base price that is used ONLY for
6   loan calculations (individual purchase prices are used for
7   buying). It matches the first property in the group on the board,
8   so this reads it straight from the board instead of storing a
9   second copy. Railways and utilities use their own mortgage value. */
10 int groupBasePrice(GameState game[], PropertyGroup group)
11 {
12     int i;
13
14     for(i = 0; i < BOARD_SIZE; i++)
15     {
16         if(game[0].board[i].type == PROPERTY &&
17            game[0].board[i].property.group == group)
18         {
19             return game[0].board[i].property.purchasePrice;
20         }
21     }
22
23     return 0;
24 }
25
26 int totalEligibleCollateral(GameState game[], int playerIndex)
27 {
28     int i;
29     int total;
30     int group;
31     int mult;
32
33     total = 0;
34
35     for(i = 0; i < BOARD_SIZE; i++)
36     {
37         if(game[0].board[i].type != PROPERTY &&
38            game[0].board[i].type != RAILWAY &&
39            game[0].board[i].type != UTILITY)
40         {
41             continue;
42         }
43
44         if(game[0].board[i].property.owner != playerIndex)
45             continue;
46
47         if(game[0].board[i].property.mortgaged)
48             continue;
49
50         if(game[0].board[i].property.loanLocked)
51             continue;
52
53         /* Rule-LK 34: a booming group's collateral is worth 15% more,
54          a declining group's 10% less. Colour property collateral
55          is based on the group base price (not the individual
56          purchase price - that is only used for buying). */
57         group = groupOf(game, i);
58
59         mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);
60
61         if(game[0].board[i].type == PROPERTY)
62             total += (groupBasePrice(game, group) * mult) / 100;
63         else
64             total += (game[0].board[i].property.mortgageValue * mult) / 100;
65     }
66
67     return total;
68 }
69
70 /* Rule-LK 2: max loan is 75% of eligible collateral. */
71 int calculateMaxLoan(GameState game[], int playerIndex)
72 {
73     return (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100;
74 }
75
76 void obtainLoan(GameState game[], int playerIndex)
77 {
78     int i;
79     int maxLoan;
80     char loanBuf[32];

```

```

81
82     if(game[0].players[playerIndex].loan.active)
83         return; /* only one active loan allowed at a time */
84
85     maxLoan = calculateMaxLoan(game, playerIndex);
86
87     if(maxLoan <= 0)
88         return;
89
90     game[0].players[playerIndex].loan.active = 1;
91     game[0].players[playerIndex].loan.amount = maxLoan;
92     game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;
93     game[0].players[playerIndex].loan.remainingRounds = LOAN_DURATION_ROUNDS;
94
95     receiveMoney(game, playerIndex, maxLoan);
96
97     printf("\n%s obtained a secured loan.\n", game[0].players[playerIndex].name);
98
99     formatLKR(maxLoan, loanBuf);
100    printf("Loan Amount : LKR %s.\n", loanBuf);
101    printf("Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
102    printf("Duration : %d Rounds\n", LOAN_DURATION_ROUNDS);
103
104    printf("Collateral :\n");
105
106    for(i = 0; i < BOARD_SIZE; i++)
107    {
108        if(game[0].board[i].type != PROPERTY &&
109           game[0].board[i].type != RAILWAY &&
110           game[0].board[i].type != UTILITY)
111        {
112            continue;
113        }
114
115        if(game[0].board[i].property.owner == playerIndex &&
116           !game[0].board[i].property.mortgaged &&
117           !game[0].board[i].property.loanLocked)
118        {
119            game[0].board[i].property.loanLocked = 1;
120            printf("%s\n", game[0].board[i].name);
121        }
122    }
123 }
124
125 void repayLoan(GameState game[], int playerIndex, int amount)
126 {
127     int i;
128
129     if(!game[0].players[playerIndex].loan.active)
130         return;
131
132     if(amount > game[0].players[playerIndex].loan.amount)
133         amount = game[0].players[playerIndex].loan.amount;
134
135     if(game[0].players[playerIndex].cash < amount)
136         amount = game[0].players[playerIndex].cash;
137
138     payMoney(game, playerIndex, amount);
139     game[0].players[playerIndex].loan.amount -= amount;
140
141     printf("\n%s repaid LKR %d towards their loan.\n",
142           game[0].players[playerIndex].name, amount);
143
144     printf("Outstanding Balance : LKR %d\n",
145           game[0].players[playerIndex].loan.amount);
146
147     if(game[0].players[playerIndex].loan.amount <= 0)
148     {
149         resetLoan(game, playerIndex);
150
151         for(i = 0; i < BOARD_SIZE; i++)
152         {
153             if(game[0].board[i].property.owner == playerIndex)
154                 game[0].board[i].property.loanLocked = 0;
155         }
156
157         printf("%s has fully repaid the loan. Collateral released.\n",
158               game[0].players[playerIndex].name);
159     }
160 }
161
162 /* Rule-LK 5: landing on the Bank offers repaying part, repaying in

```

```

163     full, or (if no loan yet) obtaining one. With an existing loan a
164     player can also extend the term, refinance at the current rate, or
165     borrow more against newly acquired collateral. Only one
166     transaction happens per visit. */
167 void handleBankVisit(GameState game[], int playerIndex)
168 {
169     printf("\n%s landed on Bank of Ceylon.\n", game[0].players[playerIndex].name);
170
171     if(game[0].players[playerIndex].loan.active)
172     {
173         if(wantsToRepayLoan(game, playerIndex))
174         {
175             int repayAmount;
176
177             /* every strategy's wantsToRepayLoan() only triggers once
178              cash can clear the whole loan, so always repay in full */
179             repayAmount = game[0].players[playerIndex].loan.amount;
180
181             repayLoan(game, playerIndex, repayAmount);
182             return;
183         }
184
185         if(wantsIncreaseLoan(game, playerIndex))
186         {
187             increaseLoan(game, playerIndex);
188             return;
189         }
190
191         if(wantsExtendLoan(game, playerIndex))
192         {
193             extendLoan(game, playerIndex);
194             return;
195         }
196
197         if(wantsRefinance(game, playerIndex))
198         {
199             refinanceLoan(game, playerIndex);
200             return;
201         }
202
203         return;
204     }
205
206     if(wantsLoan(game, playerIndex))
207     {
208         obtainLoan(game, playerIndex);
209     }
210 }
211
212 /* Rule-LK 5: a player with an active loan may borrow more against
213 collateral they've acquired since the loan was taken out. */
214 void increaseLoan(GameState game[], int playerIndex)
215 {
216     int i;
217     int availableCollateral;
218     int additional;
219
220     if(!game[0].players[playerIndex].loan.active)
221         return;
222
223     availableCollateral = 0;
224
225     for(i = 0; i < BOARD_SIZE; i++)
226     {
227         if(game[0].board[i].type != PROPERTY &&
228            game[0].board[i].type != RAILWAY &&
229            game[0].board[i].type != UTILITY)
230         {
231             continue;
232         }
233
234         if(game[0].board[i].property.owner != playerIndex)
235             continue;
236
237         if(game[0].board[i].property.mortgaged)
238             continue;
239
240         if(game[0].board[i].property.loanLocked)
241             continue;
242
243         if(game[0].board[i].type == PROPERTY)
244         {

```

```

245         availableCollateral +=
246             (groupBasePrice(game, game[0].board[i].property.group) *
247                 modifierMultiplier(game, MOD_MARKET_MORTGAGE,
248                     game[0].board[i].property.group, -1)) / 100;
249     }
250     else
251     {
252         availableCollateral += game[0].board[i].property.mortgageValue;
253     }
254 }
255
256 additional = (availableCollateral * LOAN_COLLATERAL_PERCENT) / 100;
257
258 if(additional <= 0)
259     return;
260
261 game[0].players[playerIndex].loan.amount += additional;
262
263 for(i = 0; i < BOARD_SIZE; i++)
264 {
265     if(game[0].board[i].type != PROPERTY &&
266         game[0].board[i].type != RAILWAY &&
267         game[0].board[i].type != UTILITY)
268     {
269         continue;
270     }
271
272     if(game[0].board[i].property.owner == playerIndex &&
273         !game[0].board[i].property.mortgaged &&
274         !game[0].board[i].property.loanLocked)
275     {
276         game[0].board[i].property.loanLocked = 1;
277     }
278 }
279
280 receiveMoney(game, playerIndex, additional);
281
282 printf("\n%s increased their loan by LKR %d.\n",
283     game[0].players[playerIndex].name, additional);
284 printf("New Outstanding Balance : LKR %d\n",
285     game[0].players[playerIndex].loan.amount);
286 }
287
288 /* Rule-LK 5: extend the loan term by 10 rounds. */
289 void extendLoan(GameState game[], int playerIndex)
290 {
291     if(!game[0].players[playerIndex].loan.active)
292         return;
293
294     game[0].players[playerIndex].loan.remainingRounds += LOAN_EXTEND_ROUNDS;
295
296     printf("\n%s extended their loan by %d rounds.\n",
297         game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);
298     printf("New Duration : %d Rounds remaining\n",
299         game[0].players[playerIndex].loan.remainingRounds);
300 }
301
302 /* Rule-LK 5: refinance the loan at the current economy interest rate. */
303 void refinanceLoan(GameState game[], int playerIndex)
304 {
305     if(!game[0].players[playerIndex].loan.active)
306         return;
307
308     game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;
309
310     printf("\n%s refinanced their loan at the current rate of %d%.\n",
311         game[0].players[playerIndex].name,
312         game[0].economy.loanInterestRate);
313 }
314
315 void demolishBuildingsOn(GameState game[], int index)
316 {
317     game[0].board[index].property.houses = 0;
318     game[0].board[index].property.hotel = 0;
319 }
320
321 /* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties
322 are seized and auctioned. If nothing is left, the player is
323 declared bankrupt. */
324 void foreclose(GameState game[], int playerIndex)
325 {
326     int i;

```

```
327     int ownsAnythingLeft;
328
329     printf("\n%s has defaulted on their loan!\n", game[0].players[playerIndex].name);
330
331     ownsAnythingLeft = 0;
332
333     for(i = 0; i < BOARD_SIZE; i++)
334     {
335         if(game[0].board[i].property.owner != playerIndex)
336             continue;
337
338         if(!game[0].board[i].property.loanLocked)
339         {
340             ownsAnythingLeft = 1;
341             continue;
342         }
343
344         printf("%s (pledged collateral) transferred to the Bank.\n",
345             game[0].board[i].name);
346
347         demolishBuildingsOn(game, i);
348
349         stripOwnership(game, i);
350
351         adjustOwnedCount(game, playerIndex, i, -1);
352
353         runAuction(game, i, -1);
354     }
355
356     resetLoan(game, playerIndex);
357
358     printf("Outstanding debt cleared.\n");
359
360     if(!ownsAnythingLeft && game[0].players[playerIndex].cash <= 0)
361     {
362         game[0].players[playerIndex].bankrupt = 1;
363         printf("%s has no remaining assets and is declared BANKRUPT.\n",
364             game[0].players[playerIndex].name);
365     }
366 }
367
368 void processLoans(GameState game[])
369 {
370     int i;
371     int interest;
372
373     for(i = 0; i < MAX_PLAYERS; i++)
374     {
375         if(game[0].players[i].bankrupt)
376             continue;
377
378         if(!game[0].players[i].loan.active)
379             continue;
380
381         interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100;
382         game[0].players[i].loan.amount += interest;
383
384         game[0].players[i].loan.remainingRounds--;
385
386         printf("\n%s's loan accrued LKR %d interest. New balance : LKR %d\n",
387             game[0].players[i].name, interest, game[0].players[i].loan.amount);
388
389         if(game[0].players[i].loan.remainingRounds <= 0)
390         {
391             foreclose(game, i);
392         }
393     }
394 }
```

auction.c

```

1  #include <stdio.h>
2  #include "types.h"
3  #include "functions.h"
4
5  /* Rule-LK 19: opening bid is 50% of market value. A market decline
6     lowers the starting price by 25% (Rule-LK 32/34) - group-scoped.
7     Railways and utilities use currentMarketValue too, so their value
8     modifiers (Port Expansion, Water Shortage, ...) apply the same way
9     they do for colour-group properties. */
10 int getAskingValue(GameState game[], int propIndex)
11 {
12     int value;
13     int group;
14
15     value = currentMarketValue(game, propIndex);
16
17     group = groupOf(game, propIndex);
18
19     value = (value * modifierMultiplier(game, MOD_AUCTION_PRICE, group, -1)) / 100;
20
21     return value;
22 }
23
24 /* Rule-LK 19-23: runs an auction among all solvent players. A
25    player who declines to bid is out permanently, so the pool of
26    bidders can only shrink - guaranteeing the loop finishes.
27    sellerIndex >= 0 (Anti-Speculation Act forced sale) sends the
28    proceeds to that player; sellerIndex == -1 pays the Bank. */
29 void runAuction(GameState game[], int propIndex, int sellerIndex)
30 {
31     int active[MAX_PLAYERS];
32     int activeCount;
33     int i;
34     int currentBid;
35     int highBidder;
36     int candidateBid;
37     int safetyRounds;
38     char moneyBuf[32];
39
40     printf("\n*** AUCTION ***\n");
41     printf("Property : %s\n", game[0].board[propIndex].name);
42
43     currentBid = getAskingValue(game, propIndex) / AUCTION_OPENING_DIVISOR;
44
45     formatLKR(currentBid, moneyBuf);
46     printf("Opening Bid : LKR %s\n", moneyBuf);
47
48     highBidder = -1;
49     activeCount = 0;
50
51     for(i = 0; i < MAX_PLAYERS; i++)
52     {
53         active[i] = !game[0].players[i].bankrupt;
54
55         /* Anti-Speculation Act (Rule-LK 8): the forced seller must
56            actually part with the property, so they never bid. If the
57            seller were allowed to bid, they could win the auction back
58            and merely pay the bid to themselves (net-zero cost), which
59            defeats the forced sale. */
60         if(i == sellerIndex)
61             active[i] = 0;
62
63         if(active[i])
64             activeCount++;
65     }
66
67     safetyRounds = 0;
68
69     while(activeCount > 1 && safetyRounds < AUCTION_MAX_ROUNDS)
70     {
71         for(i = 0; i < MAX_PLAYERS; i++)
72         {
73             if(!active[i])
74                 continue;
75
76             if(activeCount <= 1)
77                 break;
78
79             candidateBid = currentBid + AUCTION_BID_INCREMENT;
80

```

```

81         if(willingToBid(game, i, propIndex, candidateBid))
82         {
83             currentBid = candidateBid;
84             highBidder = i;
85
86             formatLKR(currentBid, moneyBuf);
87             printf("%s bids LKR %s.\n", game[0].players[i].name, moneyBuf);
88         }
89         else
90         {
91             active[i] = 0;
92             activeCount--;
93
94             printf("%s withdraws.\n", game[0].players[i].name);
95         }
96     }
97
98     safetyRounds++;
99 }
100
101 /* If exactly one bidder is still active, give them one final
102    chance to bid at the next increment before awarding. */
103 if(activeCount == 1 && highBidder == -1)
104 {
105     int remaining;
106
107     remaining = -1;
108
109     for(i = 0; i < MAX_PLAYERS; i++)
110     {
111         if(active[i])
112         {
113             remaining = i;
114             break;
115         }
116     }
117
118     if(remaining != -1)
119     {
120         candidateBid = currentBid + AUCTION_BID_INCREMENT;
121
122         if(willingToBid(game, remaining, propIndex, candidateBid))
123         {
124             currentBid = candidateBid;
125             highBidder = remaining;
126
127             formatLKR(currentBid, moneyBuf);
128             printf("%s bids LKR %s.\n",
129                 game[0].players[remaining].name, moneyBuf);
130         }
131     }
132 }
133
134 if(highBidder == -1)
135 {
136     int lastActive;
137
138     /* One player never got a chance to bid (e.g. the first active
139        player declined while only two were in) - they may take the
140        property at the opening bid. */
141     lastActive = -1;
142
143     for(i = 0; i < MAX_PLAYERS; i++)
144     {
145         if(active[i])
146         {
147             lastActive = i;
148             break;
149         }
150     }
151
152     /* No one left to bid, or they decline the opening price */
153     if(lastActive == -1 || !willingToBid(game, lastActive, propIndex, currentBid))
154     {
155         printf("No bids received. %s remains with the Bank.\n",
156             game[0].board[propIndex].name);
157         return;
158     }
159
160     highBidder = lastActive;
161 }
162

```

```
163     /* Always format the final winning price so the display is correct
164        regardless of which code path set highBidder. */
165     formatLKR(currentBid, moneyBuf);
166
167     payMoney(game, highBidder, currentBid);
168
169     if(sellerIndex >= 0)
170         receiveMoney(game, sellerIndex, currentBid);
171
172     game[0].board[propIndex].property.owner = highBidder;
173
174     adjustOwnedCount(game, highBidder, propIndex, 1);
175
176     printf("%s wins the auction for LKR %s.\n",
177           game[0].players[highBidder].name, moneyBuf);
178 }
```

insurance.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "types.h"
5  #include "functions.h"
6
7  int propertyValue(GameState game[], int propIndex)
8  {
9      return currentMarketValue(game, propIndex);
10 }
11
12 /* Assumption: repair costs 30% of the property's value. The
13    assignment doesn't give an exact figure, so this is used
14    consistently everywhere repair cost is needed. */
15 int repairCost(GameState game[], int propIndex)
16 {
17     return (propertyValue(game, propIndex) * REPAIR_COST_PERCENT) / 100;
18 }
19
20 int isCovered(InsuranceType policy, DisasterType disaster)
21 {
22     if(policy == NO_INSURANCE)
23         return 0;
24
25     if(policy == BASIC_INSURANCE)
26         return (disaster == FIRE || disaster == FLOOD);
27
28     return 1; /* Comprehensive and Business Interruption cover everything */
29 }
30
31 int compensationPercent(InsuranceType policy)
32 {
33     if(policy == BASIC_INSURANCE)
34         return 80;
35
36     if(policy == COMPREHENSIVE_INSURANCE || policy == BUSINESS_INTERRUPTION)
37         return 100;
38
39     return 0;
40 }
41
42 /* Picks a disaster. Every type is equally likely at base, but Heavy
43    Monsoon boosts floods and Political Unrest boosts riots (Rule-LK 34). */
44 DisasterType pickDisaster(GameState game[])
45 {
46     int weights[5];
47     int total;
48     int i;
49     int roll;
50     DisasterType result;
51
52     weights[FIRE] = DISASTER_WEIGHT;
53     weights[FLOOD] = DISASTER_WEIGHT;
54     weights[RIOT] = DISASTER_WEIGHT;
55     weights[BUILDING_COLLAPSE] = DISASTER_WEIGHT;
56     weights[ELECTRICAL_FAILURE] = DISASTER_WEIGHT;
57
58     if(isModifierActive(game, MOD_FLOOD_RISK, -1, -1))
59         weights[FLOOD] += DISASTER_BOOST;
60
61     if(isModifierActive(game, MOD_RIOT_RISK, -1, -1))
62         weights[RIOT] += DISASTER_BOOST;
63
64     total = 0;
65
66     for(i = 0; i < 5; i++)
67         total += weights[i];
68
69     roll = rand() % total;
70
71     result = FIRE;
72
73     for(i = 0; i < 5; i++)
74     {
75         if(roll < weights[i])
76         {
77             result = (DisasterType)i;
78             break;
79         }
80     }

```

```

81     roll -= weights[i];
82 }
83
84 return result;
85 }
86
87 void purchaseInsurance(GameState game[], int playerIndex, int propIndex, InsuranceType type)
88 {
89     int value;
90     int premium;
91     int premiumPercent;
92     char premiumBuf[32];
93
94     if(type == BASIC_INSURANCE)
95         premiumPercent = PREMIUM_BASIC_PERCENT;
96     else if(type == COMPREHENSIVE_INSURANCE)
97         premiumPercent = PREMIUM_COMPREHENSIVE_PERCENT;
98     else if(type == BUSINESS_INTERRUPTION)
99         premiumPercent = PREMIUM_INTERRUPTION_PERCENT;
100    else
101        return;
102
103    value = propertyValue(game, propIndex);
104    premium = (value * premiumPercent) / 100;
105
106    /* Rule-LK 34: premiums respond to insurance modifiers. */
107    premium = (premium * modifierMultiplier(game, MOD_INSURANCE, -1, -1)) / 100;
108
109    if(game[0].players[playerIndex].cash < premium)
110        return;
111
112    payMoney(game, playerIndex, premium);
113
114    game[0].board[propIndex].property.insurance = type;
115    game[0].board[propIndex].property.insuranceRoundsLeft = INSURANCE_DURATION_ROUNDS;
116
117    printf("\n%s purchased insurance for %s.\n",
118        game[0].players[playerIndex].name,
119        game[0].board[propIndex].name);
120
121    formatLKR(premium, premiumBuf);
122    printf("Premium : LKR %s.\n", premiumBuf);
123 }
124
125 /* Any owned property can be insured (Section 2.8) - no longer just
126    developed ones. */
127 int findPropertyToInsure(GameState game[], int playerIndex)
128 {
129     int i;
130
131     for(i = 0; i < BOARD_SIZE; i++)
132     {
133         if(game[0].board[i].type != PROPERTY)
134             continue;
135
136         if(game[0].board[i].property.owner != playerIndex)
137             continue;
138
139         if(game[0].board[i].property.insurance != NO_INSURANCE)
140             continue;
141
142         if(desiredInsurance(game, playerIndex, i) != NO_INSURANCE)
143             return i;
144     }
145
146     return -1;
147 }
148
149 /* Finds a policy close to expiring so it can be renewed. */
150 int findPropertyToRenew(GameState game[], int playerIndex)
151 {
152     int i;
153
154     for(i = 0; i < BOARD_SIZE; i++)
155     {
156         if(game[0].board[i].type != PROPERTY)
157             continue;
158
159         if(game[0].board[i].property.owner != playerIndex)
160             continue;
161
162         if(game[0].board[i].property.insurance == NO_INSURANCE)

```

```

163         continue;
164
165         if(game[0].board[i].property.insuranceRoundsLeft <= INSURANCE_RENEW_AT)
166             return i;
167     }
168
169     return -1;
170 }
171
172 void handleInsuranceVisit(GameState game[], int playerIndex)
173 {
174     int propIndex;
175     InsuranceType type;
176
177     printf("\n%s landed on an Insurance square.\n",
178         game[0].players[playerIndex].name);
179
180     propIndex = findPropertyToRenew(game, playerIndex);
181
182     if(propIndex != -1)
183     {
184         type = game[0].board[propIndex].property.insurance;
185
186         purchaseInsurance(game, playerIndex, propIndex, type);
187
188         printf("The policy on %s was renewed for another %d rounds.\n",
189             game[0].board[propIndex].name, INSURANCE_DURATION_ROUNDS);
190         return;
191     }
192
193     propIndex = findPropertyToInsure(game, playerIndex);
194
195     if(propIndex == -1)
196     {
197         printf("No suitable property to insure right now.\n");
198         return;
199     }
200
201     type = desiredInsurance(game, playerIndex, propIndex);
202
203     purchaseInsurance(game, playerIndex, propIndex, type);
204 }
205
206 /* Rule-LK 11: fixes a disaster-damaged building automatically once
207    the owner can afford it. Called at the start of every turn. */
208 void tryAutoRepair(GameState game[], int playerIndex)
209 {
210     int i;
211
212     for(i = 0; i < BOARD_SIZE; i++)
213     {
214         if(game[0].board[i].type != PROPERTY)
215             continue;
216
217         if(game[0].board[i].property.owner != playerIndex)
218             continue;
219
220         if(!game[0].board[i].property.damaged)
221             continue;
222
223         if(game[0].players[playerIndex].cash >= game[0].board[i].property.repairCostOwed)
224         {
225             int cost;
226             char costBuf[32];
227
228             cost = game[0].board[i].property.repairCostOwed;
229
230             formatLKR(cost, costBuf);
231
232             payMoney(game, playerIndex, cost);
233
234             printf("\n%s repaired %. It can collect rent again.\n",
235                 game[0].players[playerIndex].name, game[0].board[i].name);
236
237             printf("Repair Cost : LKR %s.\n", costBuf);
238
239             game[0].board[i].property.damaged = 0;
240             game[0].board[i].property.repairCostOwed = 0;
241         }
242     }
243 }
244

```

```

245  /* Writes the name of a disaster into out. */
246  void disasterName(DisasterType disaster, char out[])
247  {
248      if(disaster == FIRE)
249          strcpy(out, "Fire");
250      else if(disaster == FLOOD)
251          strcpy(out, "Flood");
252      else if(disaster == RIOT)
253          strcpy(out, "Riot");
254      else if(disaster == BUILDING_COLLAPSE)
255          strcpy(out, "Building Collapse");
256      else
257          strcpy(out, "Electrical Failure");
258  }
259
260  /* Pays an insured player's compensation for a disaster and returns
261     how much they received. */
262  int payCompensation(GameState game[], int chosen, int owner, int cost)
263  {
264      int compensation;
265
266      compensation = (cost * compensationPercent(game[0].board[chosen].property.insurance)) / 100;
267
268      /* Political Unrest inflates Business Interruption claims (Rule-LK 34). */
269      if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION &&
270         isModifierActive(game, MOD_BI_CLAIMS, -1, -1))
271      {
272          compensation = (compensation * DAMAGE_REPAIR_MULTIPLIER) / 100;
273      }
274
275      receiveMoney(game, owner, compensation);
276
277      printf("Insurance Claim Approved.\n");
278      printf("Compensation Paid : LKR %d\n", compensation);
279
280      /* Business Interruption also covers 5 rounds of lost rental
281         income - the repair cost is fully paid above, but the
282         property still earns nothing for 5 rounds. */
283      if(game[0].board[chosen].property.insurance == BUSINESS_INTERRUPTION)
284      {
285          game[0].board[chosen].property.lostIncomeRoundsLeft = BI_LOST_INCOME_ROUNDS;
286          printf("%s will earn no rent for the next %d rounds "
287                "(Business Interruption).\n", game[0].board[chosen].name,
288                BI_LOST_INCOME_ROUNDS);
289      }
290
291      return compensation;
292  }
293
294  /* Rule-LK 10: every 10 rounds, one random developed property may
295     be hit by a disaster. */
296  void triggerDisaster(GameState game[])
297  {
298      int developed[BOARD_SIZE];
299      int developedCount;
300      int i;
301      int chosen;
302      DisasterType disaster;
303      int owner;
304      int cost;
305      int compensation;
306      char name[32];
307
308      developedCount = 0;
309
310      for(i = 0; i < BOARD_SIZE; i++)
311      {
312          if(game[0].board[i].type == PROPERTY &&
313             (game[0].board[i].property.houses > 0 || game[0].board[i].property.hotel))
314          {
315              developed[developedCount] = i;
316              developedCount++;
317          }
318      }
319
320      if(developedCount == 0)
321          return;
322
323      chosen = developed[rand() % developedCount];
324
325      disaster = pickDisaster(game);
326

```

```
327     owner = game[0].board[chosen].property.owner;
328
329     printf("\n*** DISASTER ***\n");
330
331     disasterName(disaster, name);
332     printf("%s occurred.\n", name);
333
334     printf("Affected Property : %s (owner : %s)\n",
335           game[0].board[chosen].name, game[0].players[owner].name);
336
337     cost = repairCost(game, chosen);
338
339     if(isCovered(game[0].board[chosen].property.insurance, disaster))
340     {
341         compensation = payCompensation(game, chosen, owner, cost);
342     }
343     else
344     {
345         compensation = 0;
346         game[0].players[owner].sufferedLoss = 1;
347
348         printf("Property was NOT insured against this disaster.\n");
349     }
350
351     game[0].board[chosen].property.repairCostOwed = cost - compensation;
352
353     if(game[0].board[chosen].property.repairCostOwed < 0)
354         game[0].board[chosen].property.repairCostOwed = 0;
355
356     if(game[0].board[chosen].property.repairCostOwed > 0)
357     {
358         game[0].board[chosen].property.damaged = 1;
359         printf("%s is damaged and stops earning rent until repaired.\n",
360               game[0].board[chosen].name);
361     }
362 }
363
364 void processInsuranceExpiry(GameState game[])
365 {
366     int i;
367
368     for(i = 0; i < BOARD_SIZE; i++)
369     {
370         if(game[0].board[i].type != PROPERTY)
371             continue;
372
373         if(game[0].board[i].property.insurance == NO_INSURANCE)
374             continue;
375
376         game[0].board[i].property.insuranceRoundsLeft--;
377
378         if(game[0].board[i].property.insuranceRoundsLeft == INSURANCE_EXPIRY_WARNING)
379         {
380             printf("\nInsurance policy on %s expires in %d rounds.\n",
381                   game[0].board[i].name, INSURANCE_EXPIRY_WARNING);
382         }
383
384         if(game[0].board[i].property.insuranceRoundsLeft <= 0)
385         {
386             printf("\nInsurance policy on %s has expired.\n",
387                   game[0].board[i].name);
388
389             game[0].board[i].property.insurance = NO_INSURANCE;
390         }
391     }
392 }
```